

Ascend 310

Application Development Guide (Mind Studio)

Issue	01
Date	2020-05-30



HUAWEI TECHNOLOGIES CO., LTD.

Copyright © Huawei Technologies Co., Ltd. 2020. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Huawei Technologies Co., Ltd.

Trademarks and Permissions



HUAWEI and other Huawei trademarks are trademarks of Huawei Technologies Co., Ltd.

All other trademarks and trade names mentioned in this document are the property of their respective holders.

Notice

The purchased products, services and features are stipulated by the contract made between Huawei and the customer. All or part of the products, services and features described in this document may not be within the purchase scope or the usage scope. Unless otherwise specified in the contract, all statements, information, and recommendations in this document are provided "AS IS" without warranties, guarantees or representations of any kind, either express or implied.

The information in this document is subject to change without notice. Every effort has been made in the preparation of this document to ensure accuracy of the contents, but all statements, information, and recommendations in this document do not constitute a warranty of any kind, express or implied.

Contents

1 Introduction.....	1
2 Getting Started.....	2
2.1 Ascend AI Software Stack.....	2
2.2 Application Scenario.....	4
2.3 Preparing the Development Environment.....	9
2.4 Building Your First AI Application.....	9
3 Tutorials.....	10
3.1 Development Procedure.....	10
3.2 Creating a Project.....	11
3.3 Implementing a Project.....	11
3.3.1 Graph Configuration, Creation, and Destroying.....	11
3.3.2 Engine Implementation.....	14
3.3.3 Data Transmission.....	16
3.3.3.1 Data Transmission Between Engines.....	16
3.3.3.2 Data Transmission from the Outside to an Engine in the Graph.....	17
3.3.3.3 Data Transmission from an Engine in the Graph to Outside.....	17
3.3.4 Data Preprocessing.....	18
3.3.5 Offline Model Inference.....	22
3.3.5.1 Overview.....	22
3.3.5.2 Model Conversion.....	23
3.3.5.3 Model Inference.....	23
3.3.5.4 AIPP.....	26
3.3.5.5 Batch Size.....	29
3.3.6 Data Postprocessing.....	30
3.3.7 Memory Management.....	32
3.3.7.1 Memory Management APIs Provided by the Native Language.....	32
3.3.7.2 Memory Management APIs Provided by Matrix.....	32
3.4 Building a Project.....	37
3.5 Running a Project.....	38
4 FAQs.....	39
4.1 What Do I Do If a Core Dump Occurs in the Multi-Thread Environment When the std::cout and printf Are Used Together?.....	39

4.2 What Do I Do If Memory Is Exhausted Due to an Excessive Engine Buffer?.....	41
4.3 How Do I Configure ai_config During Engine::Init Overloading?.....	42
4.4 What Do I Do If Data Errors Occur When the Receive Memory of Multiple Engines Correspond to the Same Memory Buffer?.....	44
4.5 How Do I View the Requirements of Offline Models on the Arrangement of Input Image Data?.....	45
4.6 How Do I Configure thread_num to Meet Multi-Channel Video Decoding Requirements?.....	45
4.7 What Do I Do If the thread_num Configuration Is Incorrect During Multi-Channel Video Decoding?.....	46
4.8 What Do I Do If the Engine Thread Fails to Exit Due to an Engine Implementation Code Error on the Host Side?.....	47
4.9 What Do I Do If the Engine Thread Fails to Exit Due to an Engine Implementation Code Error on the Device Side?.....	49
4.10 Adding a Third-Party Library.....	51
4.11 What Do I Do If Memory Allocation For Other Graphs Fails After the First Graph Is Destroyed in the Single-Process, Multi-Graph Scenario?.....	52
5 Appendix.....	53
5.1 Description of the Multi-Card Multi-Chip Scenario for Atlas 300.....	53
5.2 Description of the Multi-Card Multi-Chip Scenario for Atlas 200 DK.....	55
5.3 Change History.....	56

1 Introduction

This document describes how to develop AI applications based on ASIC products powered by the Ascend AI processor and Atlas 200 Developer Kit (DK).

Table 1-1 describes the application development modes supported by the current version. This document introduces only the Mind Studio mode.

Table 1-1 Application development modes

Mode	Tool Dependency	Reference
Mind Studio	This development mode relies on the UI of Mind Studio.	Ascend 310 Application Development Guide (Mind Studio)
CLI	This development mode does not rely on Mind Studio. Applications are developed in the background in CLI mode.	<i>Application Development Guide (CLI)</i>

2 Getting Started

[2.1 Ascend AI Software Stack](#)

[2.2 Application Scenario](#)

[2.3 Preparing the Development Environment](#)

[2.4 Building Your First AI Application](#)

2.1 Ascend AI Software Stack

To give full play to the Ascend AI processor performance and help developers efficiently write AI applications running on it, Huawei provides a complete set of development tools, including computing resources, performance optimization framework, and a wide range of functions.

Figure 2-1 Ascend AI software stack

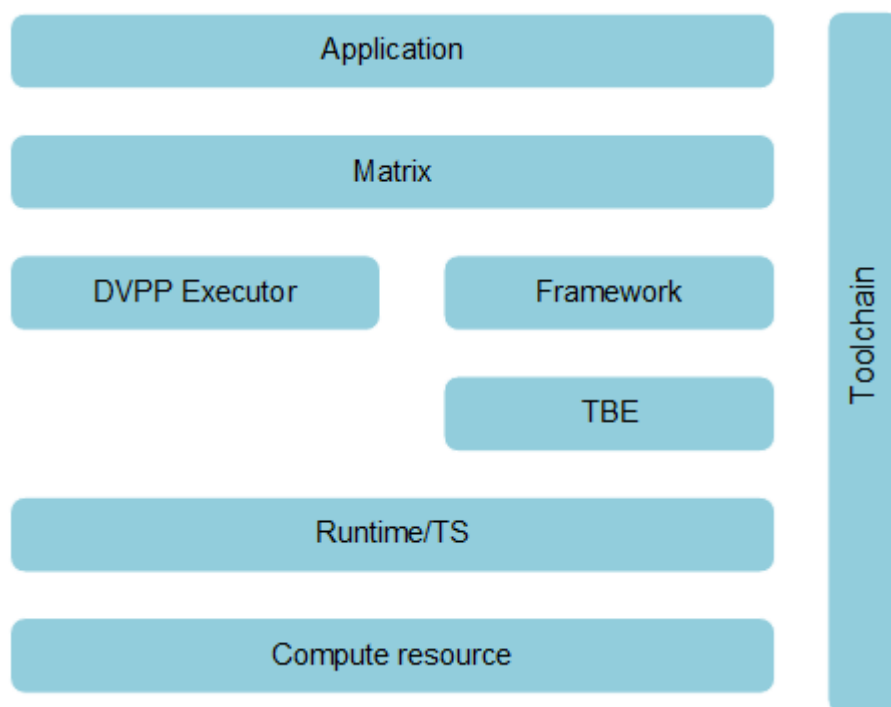


Table 2-1 Main modules

Name	Description
Matrix	Matrix implements specific functions with computing engines and orchestrates and executes computing engines through a computing engine flowchart (graph). Matrix provides APIs for engine development and process orchestration. For details about the APIs, see the <i>Matrix API Reference</i>.
DVPP Executor	DVPP Executor provides media preprocessing APIs that convert the formats and resolution of input data unsupported by the architecture, facilitating subsequent neural network computation. For details about the APIs, see the <i>DVPP API Reference</i>.
Framework	This module provides the following functions: • Offline model conversion: converts models in open source frameworks such as Caffe to the offline model supported by the Ascend AI processor. • Offline model loading and execution: AIModelManger provides an API for loading offline models to the Ascend AI processor and an API for executing offline models to implement model inference. For details about APIs, see the <i>Matrix API Reference</i>.
TBE	TBE provides the operator development capability for neural networks running on the Ascend AI processor, and builds various neural network models with diverse TBE operators. TBE offers a special refined standard operator library for neural networks. Operators in the library can be directly employed to implement high-performance neural network computing.
Runtime	Runtime is dedicated to allocating resource management channels to neural network tasks. In the process space where Runtime executes application programs, the Ascend AI processor provides application programs with various functions such as memory management, device management, stream management, event management, and kernel execution.
TS	TS is in charge of scheduling task and provide specific target tasks for the Ascend AI processor. It works with Runtime to schedule tasks and sends output data to Runtime. In this scenario, TS functions as a channel for task delivery and data backhaul.

Name	Description
Compute resource	As the basis of hardware computing for the Ascend AI processor, the compute resource layer supports deep neural network computing. • As the core of the computing power of the Ascend AI processor, AI Core mainly performs matrix-related computations of neural networks. • AI CPU controls general computations and executions such as controlling operators, scalars, and vectors. • DVPP pre-processes images and video data and provides AI Core with data in formats that meet computing requirements in specific scenarios.
Toolchain	Compatible with the Ascend AI processor, the toolchain supports application development, custom operator development, commissioning, network migration, optimization, and profiling, simplifying the development process. The toolchain provides diverse tools such as project management, compilation and commissioning, process orchestration, offline model conversion, comparison tool, log management, performance analysis tool, operator customization, and black box tool. Therefore, the toolchain offers multi-layer and multi-function services for efficient development and execution of applications on this platform.

2.2 Application Scenario

Atlas 300 Scenario

The Atlas 300 application scenario refers to the PCIe card scenario based on the Ascend AI processor, which mainly applies to data centers and edge servers, as shown in [Figure 2-2](#). The PCIe card powered by the Ascend AI processor is a dedicated acceleration card for neural network computing. Capable of multiple data precisions, it achieves higher performance and offers more robust computing power for neural networks than similar products from peer vendors.

Figure 2-2 PCIe card powered by the Ascend AI processor

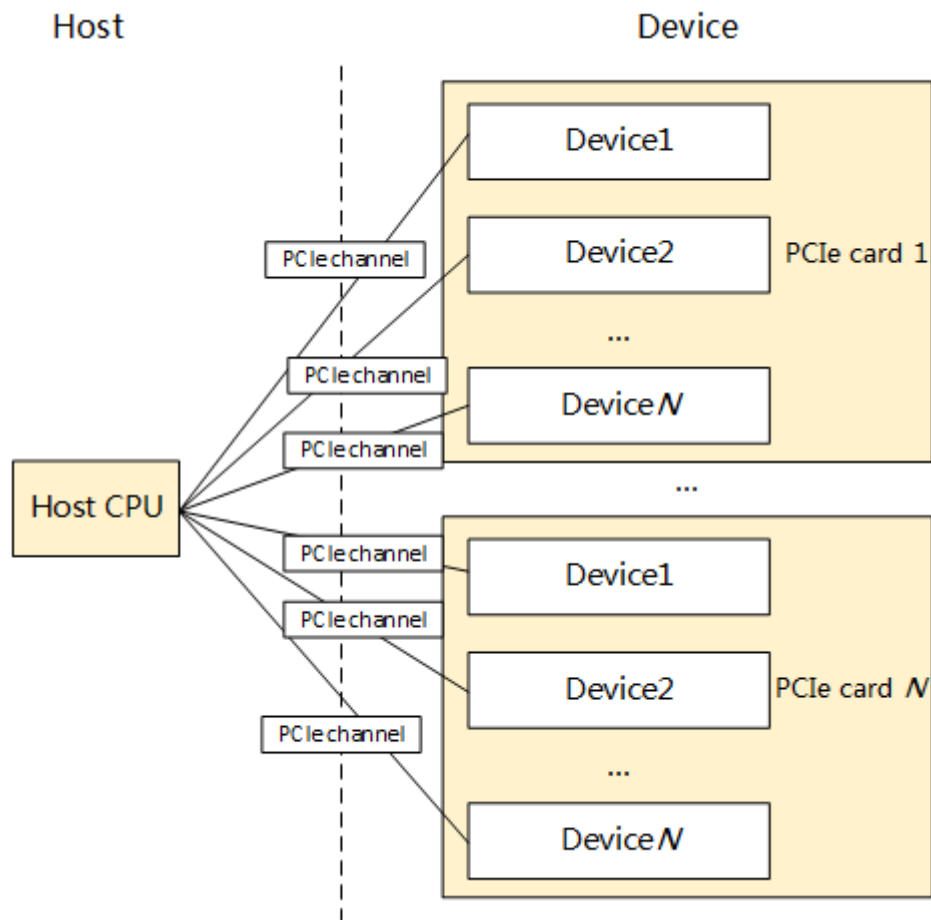


The following concepts are involved in the scenario:

- The host refers to the x86 server, Arm server, or Windows PC connected to the device. It utilizes the neural network computing capability provided by the device to complete services.
- The device refers to a hardware device powered by the Ascend AI processor and provides the host with the neural network computing capability through the PCIe interface.

The data between the host and device is transferred through the HDC driver. The hardware channel is a PCIe channel, as shown in [Figure 2-3](#).

Figure 2-3 PCIe topology



In this scenario, the entire computation is implemented by three subprocesses, including Matrix Agent (engine orchestration agent), Matrix Daemon (engine orchestration daemon), and Matrix Service (engine orchestration service), as shown in [Figure 2-4](#).

- Running on the host side, Matrix Agent controls and manages the data engine and postprocessing engine, exchanges processing data with applications, controls applications, and communicates with the device-end processing process.
- Running on the device side, Matrix Daemon creates a computation flow based on the graph configuration file, starts and manages the engine orchestration process, and destroys the computation flow and reclaims resources after the computation is complete.

- Running on the device side, Matrix Service starts and controls the preprocessing engine and model inference engine. It controls the preprocessing engine in calling the APIs of DVPP Executor to implement video and image preprocessing. Matrix Service also calls the model manager APIs to load and inference offline models.

Figure 2-4 Computation flow in the Atlas 300 scenario

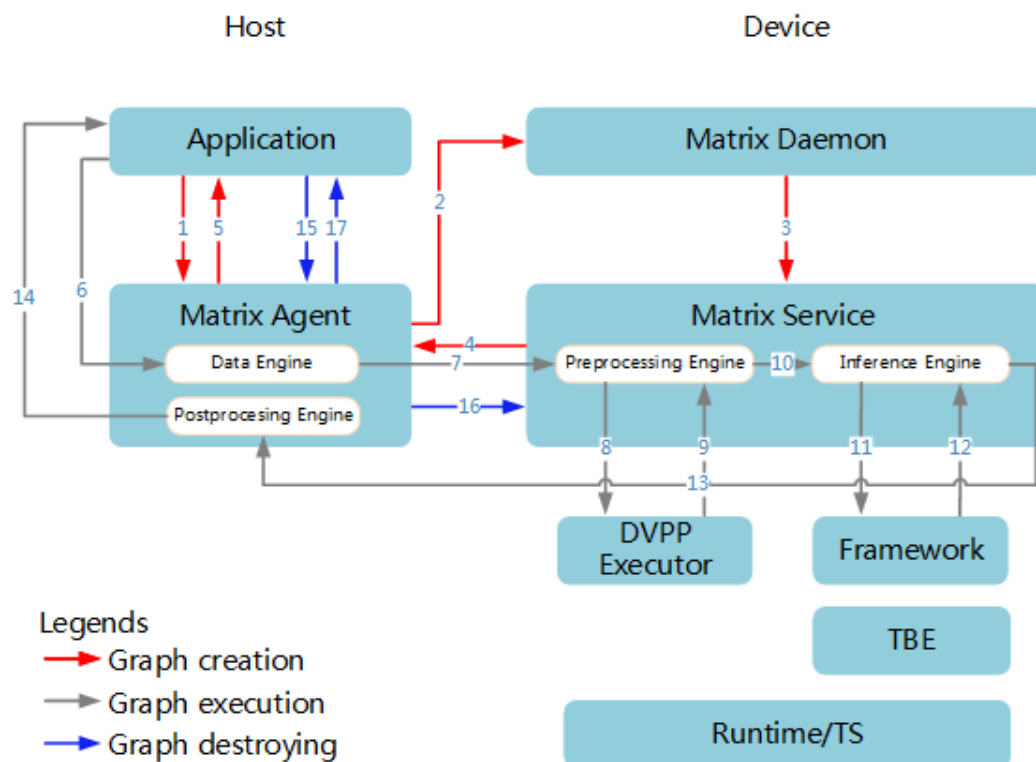


Table 2-2 Service flow description

Workflow	No.	Description
Graph creation	1-5	- A graph is created based on the graph configuration file. - The offline model file and configuration file are uploaded to the device side. - The engine is initialized. In this process, the inference engine loads models by using the Init API of the offline model manager (AIModelManager).
	6	Data is input.
	7-9	The pre-processing engine calls DVPP APIs to preprocess data, for example, encoding and decoding videos and images and cropping and resizing images.
Engine execution	10-12	The inference engine calls the Process API of the offline model manager to perform inference and computing tasks.

Workflow	No.	Description
	13-14	The inference engine calls the SendDataAPI provided by Matrix to return the inference result to DestEngine. DestEngine returns the inference result to the application using the callback function.
Graph destruction	15-17	The program is ended and the graph is destroyed.

Atlas 200 DK Scenario

The Atlas 200 DK scenario refers to the Atlas 200 Developer Kit (DK) powered by the Ascend AI processor, as shown in [Figure 2-5](#). The Atlas 200 DK opens core functions of the Ascend AI processor through peripheral interfaces on the board, which facilitates chip control and development from the outside and gives full display of the neural network processing capability of the Ascend AI processor. Therefore, the Atlas 200 DK based on the Ascend AI processor can be applied in extensive AI fields and is indispensable in mobile devices.

Figure 2-5 Atlas 200 DK powered by the Ascend AI processor



In the Atlas 200 DK scenario, the control function of the host is integrated into the developer board. Therefore, only one Matrix process is running on the device side, as shown in [Figure 2-6](#).

Figure 2-6 Computation flow in the Atlas 200 DK scenario

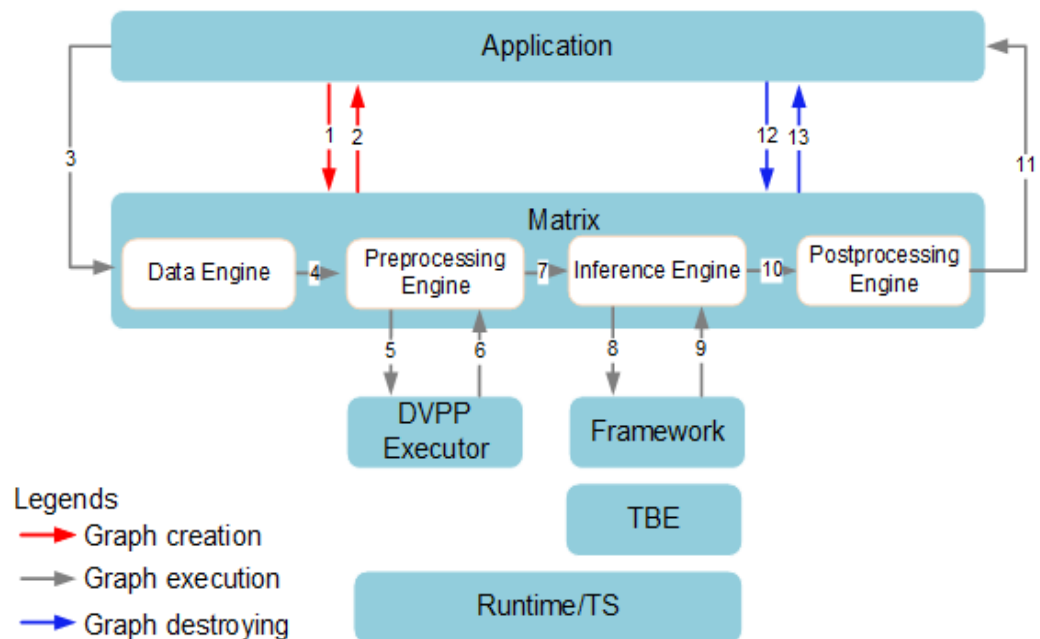


Table 2-3 Service flow description

Workflow	No.	Description
Graph creation	1-2	- A graph is created based on the graph configuration file. - The engine is initialized. In this process, the inference engine loads models by using the Init API of the offline model manager (AIModelManager).
Graph execution	3	Data is input.
	4-6	The pre-processing engine calls DVPP APIs to preprocess data, for example, encoding and decoding videos and images and cropping and resizing images.
	7-9	The inference engine calls the Process API of the AIModelManager to perform inference and computing tasks.
	10-11	The inference engine calls the SendData API provided by Matrix to return the inference result to DestEngine. DestEngie returns the inference result to the application using the callback function.
Graph destruction	12-13	The program is ended and the graph is destroyed.

2.3 Preparing the Development Environment

Before using the UI of Mind Studio to develop applications, you need to install Mind Studio (including the DDK) and configure the compilation environment (including the library package). For details, see Installation and Configuring the Compilation Environment in the *Mind Studio User Manual*.

Relevant concepts:

- Mind Studio: an AI full-stack development tool developed based on the Huawei-designed Ascend AI processor, which can be used for project management, compilation, and profiling to improve development efficiency.
- DDK: device development kit, which provides developers with a development package based on the Ascend AI processor and integrates components for AI development, such as APIs, libraries, and toolchains.
- Library package: provides library files required for compilation and running.

2.4 Building Your First AI Application

This section describes the general workflow of application development with Mind Studio by using an AI application based on the classification network as an example.

In this sample, the ResNet-18 classification network infers, computes, and classifies three png images. For details, see Quick Start > Building Your First AI App in the *Mind Studio User Manual*.

3 Tutorials

[3.1 Development Procedure](#)

[3.2 Creating a Project](#)

[3.3 Implementing a Project](#)

[3.4 Building a Project](#)

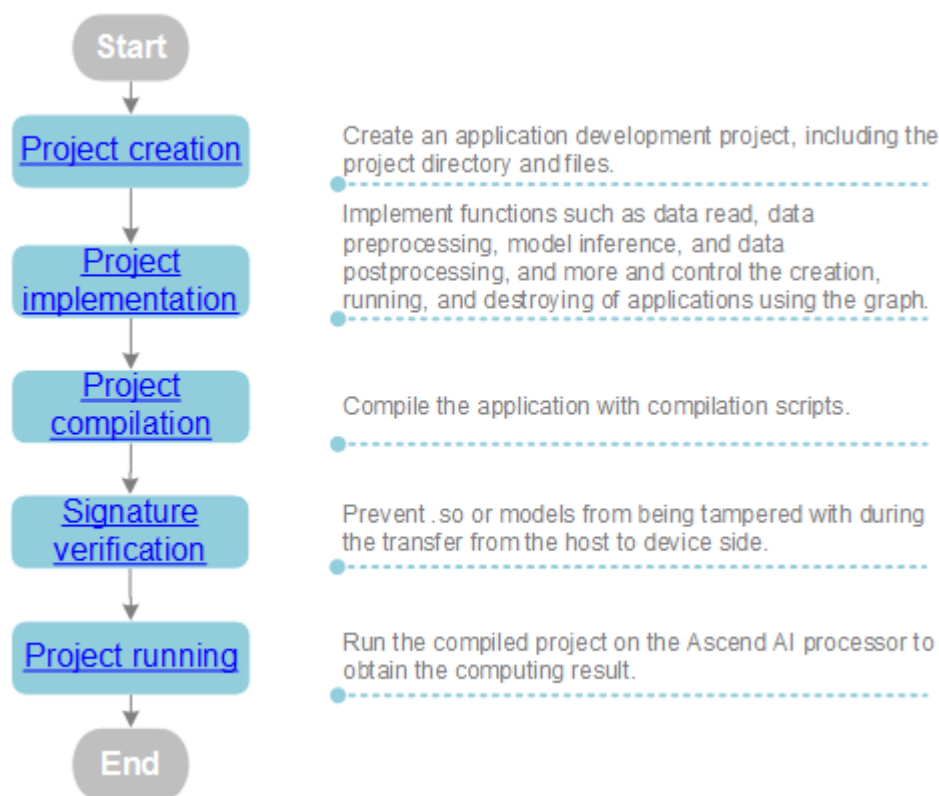
[3.5 Running a Project](#)

3.1 Development Procedure

This document describes the method of developing applications with Mind Studio and related precautions. Before application development, ensure that operations described in [2.3 Preparing the Development Environment](#) have been completed.

[Figure 3-1](#) shows the development process.

Figure 3-1 Development process



3.2 Creating a Project

Start Mind Studio and create an application development project. For details, see Tutorial > Application Development in the *Mind Studio User Manual*.

3.3 Implementing a Project

3.3.1 Graph Configuration, Creation, and Destroying

Terminology

Matrix implements specific functions with computing engines and orchestrates and executes computing engines through a computing engine flowchart (graph).

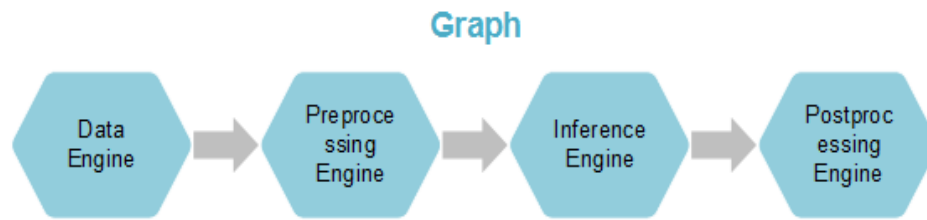
Engine

As the basic functional unit of the process, the engine supports custom implementation, including inputting image data, classifying images, and outputting the prediction result. For details, see [3.3 Implementing a Project](#).

Graph

The graph manages the flow composed of multiple engines. [Figure 3-2](#) shows the relationship between the graph and engines.

Figure 3-2 Relationship between the graph and engines



Graph configuration file

Graph information of application programs is saved as a .prototxt file, which stores engine configurations and their connections.

Similar to the .prototxt file of the Caffe model, the graph configuration file stores related configuration information through the protobuf library. The parameter format is defined by the .proto file (**include/inc/proto/graph_config.proto** in the DDK installation directory).

Creating a Graph

You can create a graph to start the engine thread and initialize engines.

1. Before creating a graph, call the **HIAI_Init** API to initialize the Host Device Communication (HDC) module, which is deployed on both host and device sides for mutual communication.

```
HIAI_StatusT HIAI_Init(uint32_t deviceId)
```

2. Call the **Graph::CreateGraph** API to create a **Graph** object.

```
static HIAI_StatusT CreateGraph(const std::string& configFile)
```

NOTE

You can also create a **Graph** object by calling **HIAI_CreateGraph**, **HIAI_CreateGraph_IdList**, **Graph::CreateGraph** (using the configuration file and writing the generated graph back to the list), and **Graph::CreateGraph** (using Protobuf).

The API calling flow includes the following subflows:

- Create a graph based on the graph configuration file **configFile**.
- Upload the offline model file and graph configuration file to the device side.
- Initialize the engines, including loading of .so files for models and engines, and dataset reading.
- Initialize the memory pool.
- Start the engine thread.

The graph configuration file **configFile** contains the following information:

- Graph information: includes the graph ID, priority, device ID, multiple engines, and connection information. For example, the sample contains three engines and two connections.
- Engine information: includes the ID, engine name, running side, number of threads, and more. It should be noted that the running side configuration determines whether the engine runs on the host or device side. The DVPP engine and the inference engine must run on the device

side, that is, on the Ascend AI processor. The data preparation engine and data postprocessing engine depend on the hardware platform. For the Atlas 200 DK running on the Ubuntu OS, the two engines run on the device side. For the Atlas 300, the two engines run on the host side.

- Engine connection information: includes the ID and sending port number of the source engine as well as the ID and sending port number of the destination engine.

The following is a sample of the graph configuration file (**configFile**).

```
graphs {
  graph_id: 100
  priority: 1
  engines {
    id: 1000
    engine_name: "SrcEngine"
    side: HOST
    thread_num: 1
  }
  engines {
    id: 1003
    engine_name: "FrameworkerEngine"
    so_name: "./libFrameworkerEngine.so"
    side: DEVICE
    thread_num: 1
    ai_config{
      items{
        name: "model_path"
        value: "./test_data/model/resnet18.om"
      }
    }
  }
}
engines {
  id: 1001
  engine_name: "DvppEngine"
  side: DEVICE
  thread_num: 1
}
engines {
  id: 1002
  engine_name: "DestEngine"
  side: HOST
  thread_num: 1
}
connects {
  src_engine_id: 1000
  src_port_id: 0
  target_engine_id: 1001
  target_port_id: 0
}
connects {
  src_engine_id: 1001
  src_port_id: 0
  target_engine_id: 1003
  target_port_id: 0
}
connects {
  src_engine_id: 1003
  src_port_id: 0
  target_engine_id: 1002
  target_port_id: 0
}
}
```

Modifying a Graph

If you need to dynamically modify engine configurations in the graph without stopping the Matrix process (for example, adding an inference engine to the graph), call the **Graph::ModifyGraph** API.

```
HIAI_StatusT ModifyGraph(const hiai::GraphUpdateConfig& config);
```

For details about the calling sample, see `modify_graph` in the *DDK Sample User Guide (CLI)*.

Destroying a Graph

After the inference result is returned, you can destroy the graph using **Graph::DestroyGraph** and end the application process.

```
static HIAI_StatusT DestroyGraph(uint32_t graphID)
```

NOTE

You can also destroy the graph by calling **HIAI_DestroyGraph**.

3.3.2 Engine Implementation

A compute engine is defined as a basic functional unit of Matrix. Its implementation can be customized (for example, input image data, classify images, and output the prediction result of image classification).

The initialization functions **Init()** and **Process()** are defined for each engine. **Init()** is optional, which is selected based on service requirements.

In [Creating a Graph](#), **Init()** automatically runs to initialize engine parameters (including memory allocation and model loading). **Process()** implements data transmission and service logic.

Header File Template

```
#include "hiaengine/api.h"
#define ENGINE_INPUT_SIZE 1
#define ENGINE_OUTPUT_SIZE 1
using hiai::Engine;

class CustomEngine : public Engine {
public:
    // Used only in the model inference engine
    HIAI_StatusT Init(const hiai::AIConfig& config, const std::vector<hiai::AIModelDescription>& model_desc)
    {};
    CustomEngine() {};
    ~CustomEngine() {};
    HIAI_DEFINE_PROCESS(ENGINE_INPUT_SIZE, ENGINE_OUTPUT_SIZE)
};
```

All engine-related header files are defined in the **include/inc/hiaengine** file in the DDK installation directory. An engine class inherits its parent class, and its name can be specified randomly.

- Engine constructor and destructor: You can modify the constructor and destructor functions of subclasses based on engine requirements.
- Engine initialization function **Init**: It is used to initialize engines during graph creation. The **ai_config** value defined in the graph configuration file will be passed to the **Init** function. You can also add custom items to the

configuration file and use them in the **Init** function. The **Init** function is typically applied to the model inference engine to start the model manager based on arguments before loading offline model files. For other engines, you can determine whether to reload this function based on the site requirements.

- **HIAI_DEFINE_PROCESS** macro: number of input and output interfaces (channels) of the engine. Matrix creates a buffer queue for each interface. The transmission relationship between engines is specified in the graph configuration file. **HIAI_DEFINE_PROCESS** is used to declare that the engine has one input interface and one output interface.

Engine Implementation Template

```
#include <memory>
#include "custom_engine.h"

HIAI_IMPL_ENGINE_PROCESS("CustomEngine", CustomEngine, ENGINE_INPUT_SIZE)
{
    // Receives data.
    std::shared_ptr<custom_type> input_arg = std::static_pointer_cast<custom_type>(arg0);
    // Implement a specific function.
    func(input_arg)
    // Sends data.
    SendData(0, "custom_type", std::static_pointer_cast<void>(input_arg));
    return HIAI_OK;
}
```

You need to overload and implement the **HIAI_IMPL_ENGINE_PROCESS** macro to achieve engine implementation. The name, corresponding class, and number of input interfaces must be specified for the engine. A maximum of 16 input interfaces are supported. After the input interface receives data, Matrix starts the **Process()** function. If multiple input interfaces are configured, each input interface triggers the **Process()** function separately after receiving data. Therefore, if service processing depends on multiple inputs, you need to implement the synchronization logic of multiple inputs.

- Data receive of the engine: Data is obtained by using pointers of the void type, such as arg0, arg1, and arg2. The number of such pointers is the same as the number of input interfaces of the engine. Up to 16 points of this type are supported. In terms of service applications, the shared pointer is used for data transmission. Because arg0 is of the void type, it needs to be converted to the required type of pointer first. **custom_type** indicates a custom type, which requires you to define the type of data to be transmitted.
- Engine implementation: Operations are performed based on the input data.
- Data transmission of the engine: After performing operations based on the input data, you need to convert the data into the void pointer type and transmit data using **SendData**. **0** indicates the output interface number, **custom_type** indicates the data type, and **input_arg** indicates the argument.

NOTE

The implementation code of the **HIAI_IMPL_ENGINE_PROCESS** macro cannot contain code logic that cannot exit (for example, infinite loop). Otherwise, engines of Matrix fail to run or be destroyed.

3.3.3 Data Transmission

Based on service applications, data transmission defined by Matrix is classified into the following types (The transmitted objects are shared pointers, but the API usage varies according to scenarios).

3.3.3.1 Data Transmission Between Engines

Matrix divides service software into the software on the host side (x86/Arm server) and the software on the device side (Ascend AI processor). Therefore, data transmission between engines is classified into inter-side transmission and intra-side transmission.

Intra-Side Transmission

Engines on the same side transmit data using **Engine::SendData**. The transferred data is the address of the shared pointer and is not copied. As the **Process()** function of engines is instantiated into threads, intra-side data transmission is implemented in memory-sharing mode. Ensure that there is no unauthorized access. Matrix recommends that the local engine does not modify the shared pointer after transmitting it to the next engine.

```
HIAI_StatusT SendData(uint32_t portId, const std::string& messageName, const shared_ptr<void>& dataPtr,
uint32_t timeOut = TIME_OUT_VALUE);
```

Inter-Side Transmission

The data to be transmitted needs to be serialized into binary data. After data transmission is complete, the data is deserialized into valid data. Therefore, you need to customize serialization and deserialization functions for custom data structures. Matrix allows you to define serialization and deserialization functions using either a common macro (**HIAI_REGISTER_DATA_TYPE**) or a high-speed macro (**HIAI_REGISTER_SERIALIZE_FUNC**). The common interface is applicable to data transmission at a rate below 256 kbit/s, whereas the high-speed interface is applicable to data transmission at a rate of 256 kbit/s or higher. The high-speed interface operates at a speed similar to that of a common interface when transmitting small memory blocks. The following is an implementation sample.

1. When data is transmitted from the host to the device side, define the data type **EngineTransNewT** to improve the transmission performance.

```
typedef struct EngineTransNew
{
    std::shared_ptr<uint8_t> trans_buff;
    uint32_t buffer_size; // buffer size
}EngineTransNewT;
```

2. Register the serialization and deserialization functions.

```
/**
 * @ingroup hiaiengine
 * @brief GetTransSearPtr, Serializes Trans data.
 * @param [in] : data_ptr Struct Pointer
 * @param [out]:struct_str Struct buffer
 * @param [out]:data_ptr Struct data pointer
 * @param [out]:struct_size Struct size
 * @param [out]:data_size Struct data size
 */
void GetTransSearPtr(void* inputPtr, std::string& ctrlStr, uint8_t*& dataPtr, uint32_t& dataLen)
{
    EngineTransNewT* engine_trans = (EngineTransNewT*)inputPtr;
    ctrlStr = std::string((char*)inputPtr, sizeof(EngineTransNewT));
```

```
dataPtr = (uint8_t*)engine_trans->trans_buff.get();
dataLen = engine_trans->buffer_size;
}

/**
 * @ingroup hiaiengine
 * @brief GetTransSearPtr,      Deserialization of Trans data
 * @param [in] : ctrl_ptr      Struct Pointer
 * @param [in] : data_ptr      Struct data Pointer
 * @param [out]:std::shared_ptr<void> Pointer to the pointer that is transmitted to the Engine
 */
std::shared_ptr<void> GetTransDearPtr(const char* ctrlPtr, const uint32_t& ctrlLen, const uint8_t*
dataPtr, const uint32_t& dataLen)
{
    EngineTransNewT* engine_trans = (EngineTransNewT*)ctrlPtr;
    std::shared_ptr<EngineTransNewT> engineTranPtr(new EngineTransNewT);
    engineTranPtr->buffer_size = engine_trans->buffer_size;
    engineTranPtr->trans_buff.reset(const_cast<uint8_t*>(dataPtr), hiai::Graph::ReleaseDataBuffer);
    return std::static_pointer_cast<void>(engineTranPtr);
}
```

3. Register the user-defined data type, serialization function, and deserialization function with the **HIAI_REGISTER_SERIALIZE_FUNC** macro. Before data transmission on the data transmission side, Matrix calls the registered serialization function to convert the structure pointer transmitted by the user into the structure buffer and data buffer. After data transmission on the data receive side, Matrix calls the registered deserialization function to convert the obtained structure buffer and data buffer into structures.

```
// Registers EngineTransNewT.
HIAI_REGISTER_SERIALIZE_FUNC("EngineTransNewT", EngineTransNewT, GetTransSearPtr,
GetTransDearPtr);
```

4. Convert the **EngineTransNewT** message into the void pointer type and send it to interface 0 using **Engine::SendData**.

```
hiai_ret = hiai::Engine::SendData(0, "EngineTransNewT", arg0);
```

5. When an engine on the device side receives data, Matrix automatically deserializes the user-defined data type **EngineTransNewT**.

```
std::shared_ptr<EngineTransNewT> input_arg = std::static_pointer_cast<EngineTransNewT>(arg0);
```

3.3.3.2 Data Transmission from the Outside to an Engine in the Graph

The **Graph::SendData** API is used to send data of the void type from the outside of the graph to the specified interface of an engine in the graph. This API is also capable of inter-side transmission and intra-side transmission, with the same requirements as those in [3.3.3.1 Data Transmission Between Engines](#). During inter-side transmission, the data to be transmitted needs to be serialized into binary data. After data transmission is complete, the data is deserialized into valid data.

```
HIAI_StatusT SendData(const EnginePortID& targetPortConfig, const std::string& messageName, const
std::shared_ptr<void>& dataPtr, const uint32_t timeout = 500)
// Configure Graph ID, Engine ID, and Port ID of the data receive side using targetPortConfig.
```

NOTE

Data or messages can also be sent from the outside to the graph through the **HIAI_C_SendData** or **HIAI_SendData** API.

3.3.3.3 Data Transmission from an Engine in the Graph to Outside

Matrix provides the template class **DataRecvInterface** of the callback function. It requires that the callback function and the output engine run on the same side.

Inter-side transmission is not allowed. Matrix transmits the data of engines in a graph to the outside of the graph using either of the following callback functions:

- **Graph::SetDataRecvFunc:**

```
static HIAI_StatusT SetDataRecvFunc(const EnginePortID& targetPortConfig, const  
std::shared_ptr<DataRecvInterface>& dataRecv);
```

 NOTE

HIAI_SetDataRecvFunc can also be used to implement the data transmission.

- **Engine::SetDataRecvFunc:**

```
HIAI_StatusT SetDataRecvFunc(const uint32_t portId, const shared_ptr<DataRecvInterface>&  
userDefineDataRecv)
```

3.3.4 Data Preprocessing

Overview

Generally, a model file supports specific data formats. If the data does not meet model requirements, input data must be preprocessed, for example, video decoding, picture decoding, or cropping, resizing, and format conversion.

You can use a third-party tool or the DVPP Executor to preprocess data. [Table 3-1](#) describes the main functions of the DVPP Executor.

Table 3-1 Main functions of the DVPP Executor

Module	Function
VDEC	Decodes the input H.264/H.265 video streams and outputs images for preprocessing in scenarios such as video recognition.
VENC	Encodes the output data of the DVPP module or the raw input YUV data, and outputs the H.264/H.265 video for direct playback and display.
JPEGD	Decodes the JPEG pictures and converts the input raw JPEG pictures into YUV data, which is used to preprocess data for neural network inference.
JPEGE	Restores the format of processed data to JPEG after JPEG pictures are processed, which is used to post-process data for neural network inference.
PNGD	Decodes PNG pictures, outputs the PNG pictures in RGB format to the Ascend AI processor for training or inference computing.
VPC	Provides functions such as converting the picture and video formats (for example, from YUV/RGB to YUV420), resizing, and cropping.

The following describes the decoding and VPC functions that are commonly used in AI application development.

Figure 3-3 shows the data preprocessing flow, which requires collaborations between Matrix, DVPP Executor, DVPP driver, and DVPP hardware.

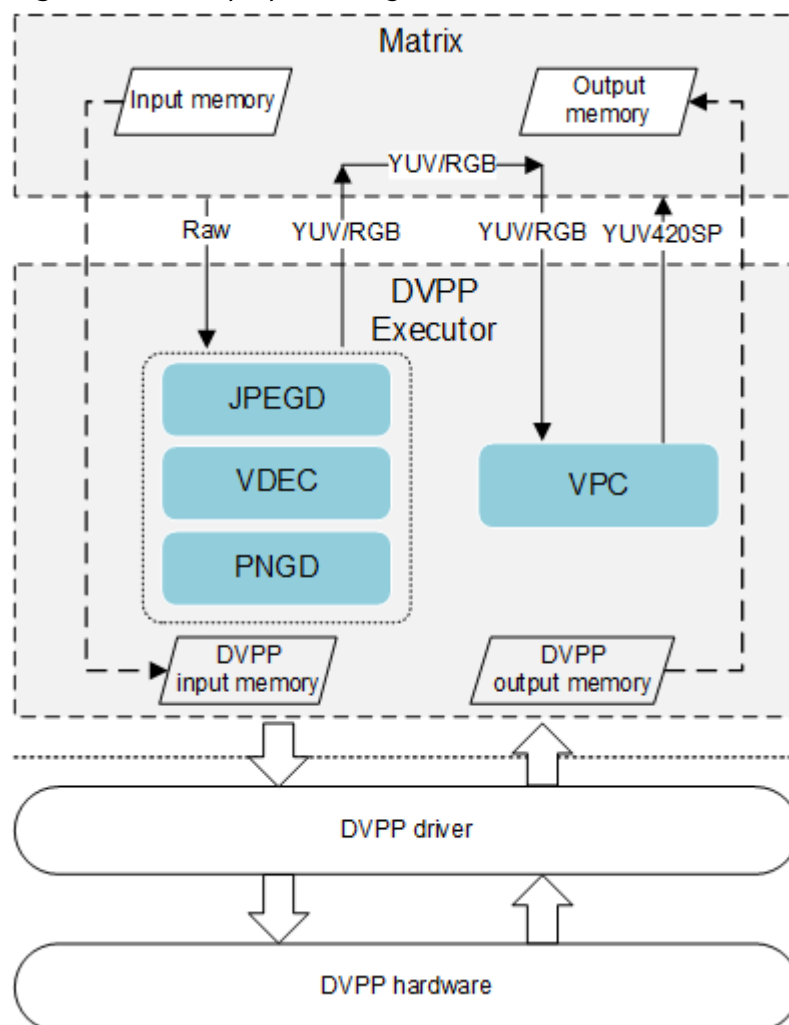
Matrix: schedules DVPP functional modules to process and manage data streams.

DVPP Executor: provides APIs for Matrix to call and set encoding/decoding and VPC parameters.

DVPP driver: manages devices and engines, and drives engine modules. The driver allocates the corresponding DVPP hardware engine based on the tasks assigned by the DVPP Executor. It reads and writes into registers of the hardware module to complete hardware initialization tasks.

DVPP hardware: a dedicated accelerator independent of other modules in the Ascend AI processor. It performs encoding, decoding, and preprocessing on images and videos.

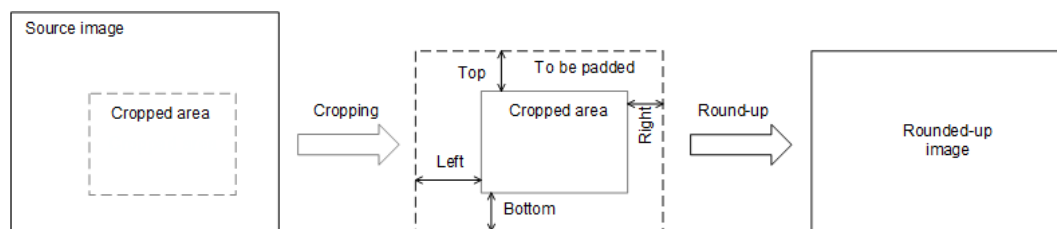
Figure 3-3 Data preprocessing flow



Matrix caches data in memory to the DVPP Executor buffer. According to the specific data format, the preprocessing engine configures parameters and transmits data through DVPP APIs. After APIs are started, DVPP Executor transfers the configuration parameters and raw data to the driver. The DVPP driver calls the functional modules of the DVPP Executor to initialize and assign tasks.

The DVPP Executor decodes images by using the JPEGD, PNGD, or VDEC module into YUV or RGB data for subsequent processing. After the decoding is complete, Matrix calls the DVPP module using the same mechanism to further convert the images into the YUV420SP format, because the YUV420SP format features low bandwidth usage. As a result, more data can be transmitted at the same bandwidth, meeting high throughput requirements of AI Core for robust computing. In addition, the VPC module can be used for image cropping and resizing. [Figure 3-4](#) shows image resizing and round-up. The DVPP Executor crops the desired proportion of the source image, performs zero padding, and retains edge features in a convolutional neural network (CNN) computing process. Zero padding is required for the top, bottom, left, and right regions. Image edges are extended in zero padding regions to generate an image that can be directly used for computation.

Figure 3-4 Image resizing



After a series of preprocessing, the image data complying with format requirements is sent to the AI Core under the control of the AI CPU for neural network computing. Meanwhile, the computing resources of the DVPP are freed and reclaimed.

VPC

The DVPP Executor provides the VPC function that converts image and video formats (for example, from YUV/RGB to YUV420), resizing, and cropping. The API calling sequence is as follows:

1. Call **CreateDvppApi** to create a DVPP API. It is used to obtain the dvppapi instance, which is equivalent to the handle of the DVPP executor and used to call the DVPP function.

```
int32_t CreateDvppApi(IDVPPAPI *&pIDVPPAPI)
```
2. Call the **DvppCtl** API to preprocess the image.

```
int32_t DvppCtl(IDVPPAPI *&pIDVPPAPI, DVPP_CTL_VPC_PROC, dvppapi_ctl_msg *MSG)
```

 - Data input format requirements: For details, see the *DVPP API Reference*. If the input format does not meet requirements, you need to convert the format.
 - Round-up requirement: DVPP is restricted by hardware during its usage. To speed up data read and write, an image's length and width must be rounded up as specified without affecting the valid region. The image's length and width are rounded up as specified by padding 0s to leftward and downward. For example, for a 300 x 300 YUV420SP_UV image, the size must be rounded up to 304 x 300 (The width should be rounded up to the nearest multiple of 16 pixels, and the height must be rounded up to the nearest multiple of 2 pixels). The valid region ranges from [0, 0] to [300, 300]. In this case, you need to pad 0s rightward to column 304.

3. Call the **DestroyDvppApi** API to release the DVPP API and close the DVPP Executor.

```
DestroyDvppApi(pidvppapi)
```

JPEG/PNGD/VDEC

The DVPP Executor decodes images and videos with the JPEGD, PNGD, and VDEC functions. The interface calling sequence is as follows:

Step 1 Create a DVPP API.

- If the JPEGD/PNGD function is used, call **CreateDvppApi** to obtain the dvppapi instance, which is equivalent to the handle of the DVPP executor and is used to call DVPP.

```
IDVPPAPI *pidvppapi = nullptr;  
int32_t ret = CreateDvppApi(pidvppapi);
```
- If the VDEC function is used, call **CreateVdecApi** to obtain the vdecapi instance.

```
int CreateVdecApi(IDVPPAPI *&pidvppapi, int singleton)
```

Step 2 Preprocess the image.

- If the JPEGD/PNGD function is used, call **DvppCtl**.

```
ret = DvppCtl(pidvppapi, DVPP_CTL_VPC_PROC, &dvppApiCtlMsg);
```
- If the VDEC function is used, call **VdecCtl**.

```
int VdecCtl(IDVPPAPI *&pidvppapi, int CMD, dvppapi_ctl_msg *MSG, int singleton)
```

Data input format requirements: For details, see the *DVPP API Reference*. If the input format does not meet requirements, you need to convert the format.

Round-up requirement: When the JPEGD, PNGD, and VDEC components of the DVPP are used to read input images, the decoded images must meet the length and width round-up requirements. In this case, you need to apply for memory for output images based on the size of the rounded-up images. For example, for a 300 x 300 YUV420SP_UV image, you need to apply for memory with the size of (304 x 300 x 3/2) bytes. Each pixel of a YUV420SP image requires a 1.5-byte storage space.

Step 3 Release the DVPP API and disable the DVPP Executor.

- If the JPEGD/PNGD function is used, call **DestroyDvppApi**.

```
DestroyDvppApi(pidvppapi)
```
- If the VDEC function is used, call **DestroyVdecApi**.

```
int DestroyVdecApi(IDVPPAPI *&pidvppapi, int singleton)
```

----End

Memory Management

Matrix provides the memory allocation API **HIAIMemory::HIAI_DVPP_DMAlloc** and memory free API **HIAIMemory::HIAI_DVPP_DFree** for the DVPP. **HIAIMemory::HIAI_DVPP_DMAlloc** is used to allocate memory that meets DVPP round-up requirements. The two APIs must be used in pairs. To prevent memory leak, you are advised to use the shared pointer to manage the allocated memory. The implementation code is as follows:

```
uint8_t* buffer = nullptr;  
HIAI_StatusT ret = hiai::HIAIMemory::HIAI_DVPP_DMAlloc(dataSize, (void*)&buffer);
```

```
std::shared_ptr<uint8_t> dataBuffer = std::shared_ptr<uint8_t>( buffer, \
[] (std::uint8_t* data) hiai::HIAIMemory::HIAI_DVPP_DFree(data);));
```

These APIs are available only on the device side. The memory allocated by **HIAIMemory::HIAI_DVPP_DMalloc** is used for high-speed data transfer from the device to the host side. Matrix is responsible for freeing memory. For details, see [3.3.7 Memory Management](#).

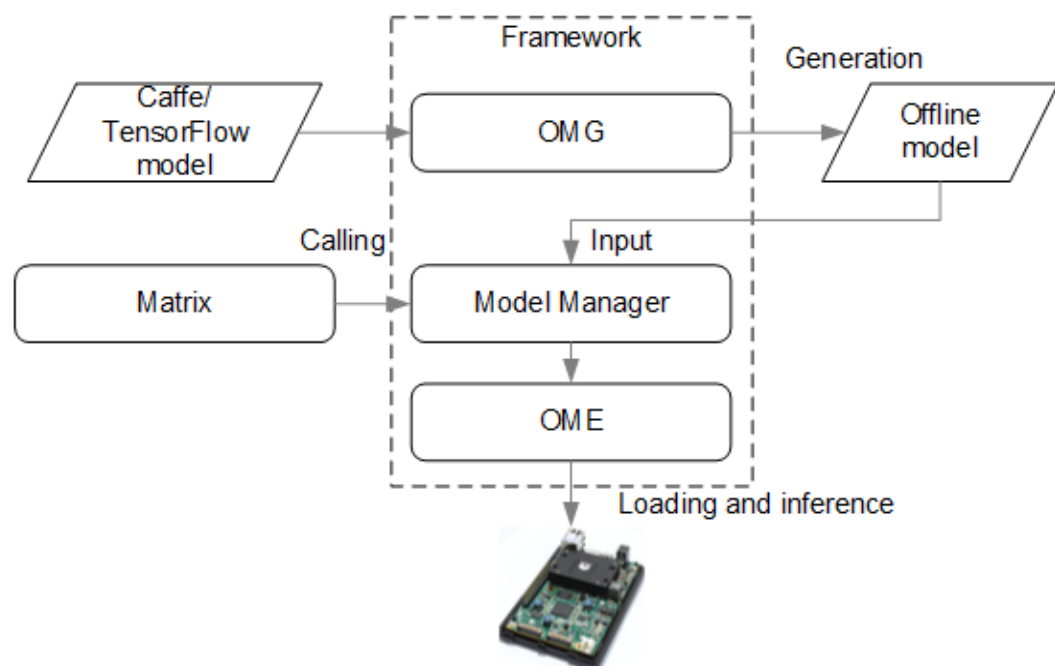
If the JPEGD/PNGD function is used, call **DvppGetOutParameter** to obtain the size of output memory of the JPEGD/PNGD module before applying for the memory.

3.3.5 Offline Model Inference

3.3.5.1 Overview

[Figure 3-5](#) shows the model inference process.

Figure 3-5 Model inference process



1. Before inference, convert a Caffe or TensorFlow model to an offline model supported by the Ascend AI processor (.om files) using the Offline Model Generator (OMG).
2. Call Model Manager in Framework through Matrix in the Ascend AI software stack to start the Offline Model Executor (OME) and load the model to the Ascend AI processor. Finally, complete model inference using the Ascend AI software stack to obtain the application output of neural network.

During model conversion, if the functions such as image cropping, format conversion, and image normalization of the AIPP module are enabled, the input data must be processed before it is used for model inference.

3.3.5.2 Model Conversion

You need to convert models in frameworks such as Caffe and TensorFlow to .om offline models supported by the Ascend AI processor. For details, see *Model Conversion* in the *Mind Studio User Manual* or the *Ascend 310Atlas 200 Model Conversion Guide*.

The name and storage path of a converted model file must be the same as those in the graph configuration file. In this way, Matrix can directly load the offline model from the storage path.

```
engines {
  id: 1003
  engine_name: "FrameworkerEngine"
  so_name: "./libFrameworkerEngine.so"
  side: DEVICE
  thread_num: 1
  ai_config{
    items{
      name: "model_path"
      value: "./test_data/model/resnet18.om" // name and path of the offline model file
    }
  }
}
```

3.3.5.3 Model Inference

A typical offline model inference and computing process includes the following stages:

Model Initialization

Reload the engine initialization function **Engine::Init** and call **AIModelManager::Init** to load offline models or perform other initialization operations.

- Load an offline model from a file:

```
AIModelManager model_mgr;
AIModelDescription model_desc;
AIConfig config;
/* Check whether the input file path is valid.
   Path: The value can contain uppercase letters, lowercase letters, digits, and underscores (_).
   File name: The value can contain uppercase letters, lowercase letters, digits, underscores (_), and
   periods (.). */
model_desc.set_path(MODEL_PATH);
model_desc.set_name(MODEL_NAME);
model_desc.set_type(0);
vector<AIModelDescription> model_descs;
model_descs.push_back(model_desc);
// AIModelManager Init
AIStatus ret = model_mgr.Init(config, model_descs);
if (SUCCESS != ret)
{
  printf("AIModelManager Init failed. ret = %d\n", ret);
  return -1;
}
```

- Load an offline model from memory:

```
AIModelManager model_mgr;
AIModelDescription model_desc;
vector<AIModelDescription> model_descs;
AIConfig config;
model_desc.set_name(MODEL_NAME);
model_desc.set_type(0);
```

```
char *model_data = nullptr;
uint32_t model_size = 0;
ASSERT_EQ(true, Utils::ReadFile(MODEL_PATH.c_str(), model_data, model_size));
model_desc.set_data(model_data, model_size); // The value of model_size must be the same as
the actual size of the model.
model_desc.set_size(model_size);
AIStatus ret = model_mgr.Init(config, model_descs);
if (SUCCESS != ret)
{
    printf("AIModelManager Init failed. ret = %d\n", ret);
    return -1;
}
}
```

Memory Management for Model Running

The system automatically allocates memory required for model running, including the working memory for storing input and output data and the weight memory for storing weight data. If you need to specify memory (for example, automatic reading and writing of the weight memory is used to dynamically update the weight online), you can call **AIModelManager::Init** when reloading **Engine::Init**.

For details about the API calling flow and example, see "Memory Configuration APIs" in the *Ascend 310 Matrix API Reference*.

Requirements of Model Inference for Input Images

- Generally, a Caffe model requires that input image data be arranged in NCHW mode, while a TensorFlow requires that input image data be arranged in NHWC mode. However, when a converted offline model is used for inference, the input image data must meet the following requirements: If AIPP is not enabled for model conversion, the input image data must be arranged in NCHW format. Therefore, the source image data arranged in NHWC mode must be automatically converted the NCHW format. If **3.3.5.4 AIPP** is enabled for model conversion, the input RGB888 data must be converted to the NHWC format. There are no specific format requirements on the input YUV data.

NOTE

For details about the requirements for input image data, see "FAQs."

- When AIPP is disabled for model conversion, the format of input image data for offline model inference must be the same as that used during model training, for example, RGB. When **3.3.5.4 AIPP** is enabled for model conversion, the following image formats are supported: YUV420SP_U8, XRGB8888_U8, RGB888_U8, and YUV400_U8.
- For the same model, if images in the same dataset are used for inference, their sizes must be the same.

Creation of Input and Output Tensors

Pay attention to the following points when setting input and output tensors for model inference:

- Matrix defines the **IAITensor** class that manages the input and output matrices for model inference. For ease of use, Matrix derives **AISimpleTensor** and **AINeuralNetworkBuffer** based on the **IAITensor** class.

- The input and output memory for model inference are allocated using **HIAI_DMAlloc**, which reduces one memory copy. For details about memory management, see [3.3.7 Memory Management](#).

The implementation code for input and output tensors is as follows:

1. Call **AIModelManager::GetModelIOTensorDim** to obtain the input and output tensor descriptions of the inference model.

```
std::vector<hiiai::TensorDimension> inputTensorDims;
std::vector<hiiai::TensorDimension> outputTensorDims;
ret = modelManager->GetModelIOTensorDim(modelName, inputTensorDims, outputTensorDims);
```

2. Configure the input tensor. If there are multiple input tensors, create and set them one by one.

```
std::shared_ptr<hiiai::AISimpleTensor> inputTensor =
std::shared_ptr<hiiai::AISimpleTensor>(new hiiai::AISimpleTensor());
inputTensor->SetBuffer (<memory address of the input data>, <length of the input data>);
inputTensorVec.push_back(inputTensor);
```

NOTE

You can also configure the input tensor by calling **AITensorFactory::CreateTensor** or **AIModelManager::CreateInputTensor**.

3. Call **AITensorFactory::CreateTensor** to configure the output tensor.

NOTE

You can also configure the output tensor by calling **AIModelManager::CreateOutputTensor**.

```
for (uint32_t index = 0; index < outputTensorDims.size(); index++) {
hiiai::AITensorDescription outputTensorDesc = hiiai::AINeuralNetworkBuffer::GetDescription();
uint8_t* buf = (uint8_t*)HIAI_DMAlloc(outputTensorDims[index].size);
.....
std::shared_ptr<hiiai::AITensor> outputTensor = hiiai::AITensorFactory::GetInstance()->CreateTensor(
outputTensorDesc, buf, outputTensorDims[index].size);
outputTensorVec.push_back(outputTensor);
}
```

Model Inference and Computing

Call **AIModelManager::Process** to perform model inference.

```
virtual AIStatus Process(AIContext &context, const std::vector<std::shared_ptr<IAITensor>> &in_data,
std::vector<std::shared_ptr<IAITensor>> &out_data, uint32_t timeout);
```

Other precautions for model inference:

- Matrix supports either synchronous inference and asynchronous inference. Synchronous inference is used by default. You can configure a callback function for model management by calling **AIModelManager::SetListener**, to implement asynchronous inference.
- To achieve inference for multiple models, before calling the **Process ()** function, call the **AddPara** function to set the name of each model.

```
ai_context.AddPara("model_name", modelName);// Multiple models. You must set a name for each
model separately.
ret = ai_model_manager->Process(ai_context,
inDataVec, outDataVec, 0);
```

3.3.5.4 AIPP

Function Description

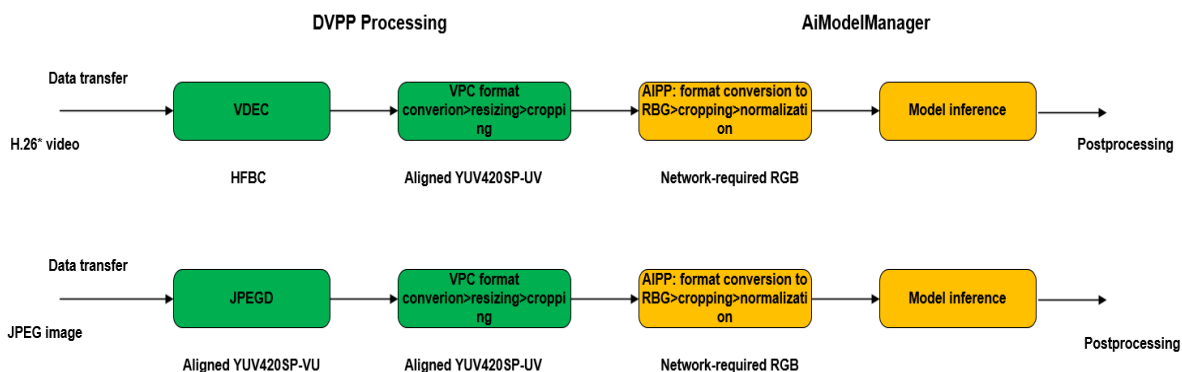
After DVPP pre-processing as described in [3.3.4 Data Preprocessing](#), sub-modules of DVPP pose many restrictions on output images to ensure the processing speed and memory usage. For example, the length and width of output images must be rounded up as specified, and the output format must be **YUV420SP**. However, the model input is usually in RGB or BGR format, and the sizes of the input images are different.

Therefore, AI Pre-processing (AIPP) is introduced for image pre-processing before model inference including image resizing, color space conversion (CSC), and mean subtraction and multiplication factor (pixel changing). All these functions are implemented by the AI Core.

AIPP supports the following image input formats: YUV420SP_U8, XRGB8888_U8, RGB888_U8, and YUV400_U8.

The following uses JPEG image input and H26* video input as an example to describe the processing flow, as shown in [Figure 3-6](#).

Figure 3-6 Processing flow of video and image inputs



For details about how to configure AIPP, see Model Conversion in the *Mind Studio User Manual* or the *Ascend 310 Model Conversion Guide*. The following details dynamic AIPP and static AIPP.

Static AIPP

During model conversion, set the AIPP mode to static and AIPP parameters. After the model is generated, the AIPP parameter values are saved in the offline model. The AIPP parameter configurations are the same in each phase of model inference.

NOTE

If the static AIPP mode is used, the same AIPP parameter configurations also apply when multiple images are processed at a time for inference.

For example, the model requires the input of a 300 x 300 RGB image. After DVPP APIs are called for processing (such as JPEG decoding and resizing), DVPP outputs

a 384 x 304 image YUV420SP_UV image (the valid region size is 300 x 300, and 0s are padded to leftward and downward).

The static AIPP configurations are as follows:

```
aipp_op{
# Set AIPP to static mode.
aipp_mode: static
# Enable image cropping.
crop: true
# Set the format and size for an input image.
input_format : YUV420SP_U8
src_image_size_w : 384
src_image_size_h : 304
# Set the start coordinates for cropping. The width and height of the cropped region are in line with the
model input by default.
load_start_pos_h : 0
load_start_pos_w : 0

# Enable format conversion. The conversion matrix converts YUV420SP_UV to RGB888.
csc_switch : true
matrix_r0c0 : 298
matrix_r0c1 : 516
matrix_r0c2 : 0
matrix_r1c0 : 298
matrix_r1c1 : -100
matrix_r1c2 : -208
matrix_r2c0 : 298
matrix_r2c1 : 0
matrix_r2c2 : 409
input_bias_0 : 16
input_bias_1 : 128
input_bias_2 : 128

# Enable data normalization and configures the mean value and the reciprocal of variance.
mean_chn_0 : 125
mean_chn_1 : 125
mean_chn_2 : 125
var_reci_chn_0 : 0.0039
var_reci_chn_1 : 0.0039
var_reci_chn_2 : 0.0039
}
```

Dynamic AIPP

In dynamic AIPP mode, AIPP parameters specified in the offline model are not used. Instead, values of dynamic AIPP parameters are specified during inference. Dynamic AIPP is used when preprocessing parameters have to be changed based on service requirements. For example, cameras use different normalization parameters, and the input image format must be compatible with YUV420 and RGB.

NOTE

In dynamic AIPP mode, batches use separate AIPP parameter configurations (such as crop and resize) defined by the dynamic parameter structure. For details about the dynamic parameter structure, see the *Model Conversion Guide*.

When using the dynamic AIPP function, pay attention to the following points:

1. Set the AIPP mode to dynamic for model conversion.
2. Set AIPP parameters before the model inference engine calls **AIModelManager::Process** to perform model inference. For details, see the following code:

```
// Define constants.
const static int16_t DEFAULT_MEAN_VALUE_CHANNEL_0 = 104;
const static int16_t DEFAULT_MEAN_VALUE_CHANNEL_1 = 117;
const static int16_t DEFAULT_MEAN_VALUE_CHANNEL_2 = 123;
const static int16_t DEFAULT_MEAN_VALUE_CHANNEL_3 = 0;
const static float DEFAULT_REC1_VALUE = 1.0;
const static int32_t CROP_START_LOCATION = 0;
const static int32_t WIDTH_ALIGN = 16;
const static int32_t HEIGHT_ALIGN = 2;
// Specify AIPP parameters.
stringstream ss;
ss << batchSize_;
string batchSizeStr = ss.str();
hiiai::AITensorDescription dynamicParam =
hiiai::AippDynamicParaTensor::GetDescription(batchSizeStr);
shared_ptr<hiiai::AITensor> tmpTensor = hiiai::AITensorFactory::GetInstance()-
>CreateTensor(dynamicParam);
shared_ptr<hiiai::AippDynamicParaTensor> dynamicParamTensor =
std::static_pointer_cast<hiiai::AippDynamicParaTensor>(tmpTensor);

// if there are multiple input, we can set multiple input or input edge, default 0
dynamicParamTensor->SetDynamicInputEdgeIndex(0);
dynamicParamTensor->SetDynamicInputIndex(INPUT_INDEX_0);

hiiai::AippInputFormat inputImageFormat = hiiai::YUV420SP_U8;
hiiai::AippModelFormat modelImageFormat = hiiai::MODEL_BGR888_U8;
// set input format
dynamicParamTensor->SetInputFormat(inputImageFormat);

//set csc params if csc switch is true
dynamicParamTensor->SetCscParams(inputImageFormat, modelImageFormat, hiiai::JPEG);

int32_t cropHeight = 224;
int32_t cropWidth = 224;
int32_t inputImageWidth = ceil(cropWidth * 1.0 / WIDTH_ALIGN) * WIDTH_ALIGN;
int32_t inputImageHeight = ceil(cropHeight * 1.0 / HEIGHT_ALIGN) * HEIGHT_ALIGN;

//If use image preprocess, set src image size
dynamicParamTensor->SetSrcImageSize(inputImageWidth, inputImageHeight);

//Every image of a batch can set these properties below independently
for (int batchSize = 0; batchSize < batchSize_; batchSize++) {
    //set default crop, we can set it customize
    dynamicParamTensor->SetCropParams(true,
        CROP_START_LOCATION,
        CROP_START_LOCATION,
        cropWidth,
        cropHeight,
        batchSize);

    //set default mean value, we can set it customize
    dynamicParamTensor->SetDtcPixelMin(DEFAULT_MEAN_VALUE_CHANNEL_0,
        DEFAULT_MEAN_VALUE_CHANNEL_1,
        DEFAULT_MEAN_VALUE_CHANNEL_2,
        DEFAULT_MEAN_VALUE_CHANNEL_3,
        batchSize);

    //set default dtcPixelVarReci value, we can set it customize
    dynamicParamTensor->SetPixelVarReci(DEFAULT_REC1_VALUE,
        DEFAULT_REC1_VALUE,
        DEFAULT_REC1_VALUE,
        batchSize);
}

hiiaiRet = aiModelManager_->SetInputDynamicAIPP(inputDataVec_, dynamicParamTensor);
```

```
if (hiaiRet != haii::SUCCESS) {
    HIAI_ENGINE_LOG(HIAI_IDE_ERROR, "[MindInferenceEngine] Failed to set input dynamic aipp.");
    return HIAI_ERROR;
}
```

3.3.5.5 Batch Size

The batch size indicates the number of pictures processed by the model at a time. Generally, the batch size is determined by the model (that is, **Static Batch Size**). Alternatively, the batch size can be specified by the user (that is, **Dynamic Batch Size**).

Static Batch Size

For model conversion with static batch size, the batch size is the value of **N** in the network model. This mode is applicable to the scenario where the number of images per batch is fixed.

Dynamic Batch Size

For model conversion with dynamic batch size, the batch size can be specified during inference. Pay attention to the following points:

1. During model conversion, set the image processing mode to dynamic batch size and set the batch size choices.
2. Before the model inference engine calls **AIModelManager::Process** for model inference, specify the number of images processed per batch.

```
hiaiRet = aiModelManager->SetDynamicBatchNumber(aiContext,inputDataVec_,1); // Set the
batch size. If the argument is not among the preset batch size choices, the maximum preset batch
size closest to the argument applies.
if (hiaiRet != haii::SUCCESS) {
    HIAI_ENGINE_LOG(HIAI_IDE_ERROR, "[MindInferenceEngine] Failed to set dynamic batch.");
    return HIAI_ERROR;
}
```

3. The input memory size of the inference API must be the same as the memory size required by the largest batch size choice of the model. Otherwise, the input memory transferred to the inference API must be padded.

```
char* Util::ReadBinFile(std::shared_ptr<std::string> file_name,
    uint32_t* file_size, int32_t batchSize, bool& isDMalloc)
{
    std::filebuf *pbuf;
    std::ifstream filestr;
    size_t size;
    filestr.open(file_name->c_str(), std::ios::binary);
    if (!filestr)
    {
        return NULL;
    }

    pbuf = filestr.rdbuf();
    size = pbuf->pubseekoff(0, std::ios::end, std::ios::in)*batchSize;
    pbuf->pubseekpos(0, std::ios::in);
    char * buffer = nullptr;
    isDMalloc = true;
    HIAI_StatusT getRet = haii::HIAIMemory::HIAI_DMAlloc(size, (void*)&buffer, 10000,
        haii::HIAI_MEMORY_ATTR_MANUAL_FREE);
    if ((getRet != HIAI_OK) || (buffer == nullptr)) {
        buffer = new(std::nothrow) char[size];
        if (buffer != nullptr) {
            isDMalloc = false;
        }
    }
}
```

```
}  
  
pbuf->sgetn(buffer, size);  
*file_size = size;  
filestr.close();  
return buffer;  
}
```

4. The output result memory of the model supporting multiple batch size choices is allocated based on the maximum batch size choice. Therefore, during the post-processing of the inference result, only the actual image inference result is retained.

```
for(int idx=0;idx<batch_num;++idx){  
    Output out;  
    out.size = result_tensor->GetSize()/kMax_Batch_size;  
    out.data = std::shared_ptr<uint8_t>(new (nothrow) uint8_t[out.size],  
                                       std::default_delete<uint8_t[]>());  
    if (out.data == nullptr) {  
        HIAI_ENGINE_LOG(HIAI_ENGINE_RUN_ARGS_NOT_RIGHT,  
                        "dealing results: new array failed");  
        continue;  
    }  
    errno_t mem_ret = memcpy_s(out.data.get(), out.size,  
                              result_tensor->GetBuffer()+out.size * idx,  
                              out.size);  
    // memory copy failed, skip this result  
    if (mem_ret != EOK) {  
        HIAI_ENGINE_LOG(HIAI_ENGINE_RUN_ARGS_NOT_RIGHT,  
                        "dealing results: memcpy_s() error=%d", mem_ret);  
        continue;  
    }  
    image_handle->inference_res.emplace_back(out);  
}
```

3.3.6 Data Postprocessing

The result matrix of model inference is stored in the **IAITensor** object as the memory and description information. You need to parse the memory information into valid output based on the actual output format (data type and data sequence) of the model.

```
/* Parse the inference result. */  
for (uint32_t index = 0; index < outputTensorVec.size(); index++) {  
    shared_ptr<hiiai::AINeuralNetworkBuffer> resultTensor =  
        std::static_pointer_cast<hiiai::AINeuralNetworkBuffer>(outputTensorVec[i]);  
    // resultTensor->GetNumber() -- N  
    // resultTensor->GetChannel() -- C  
    // resultTensor->GetHeight() -- H  
    // resultTensor->GetWidth() -- W  
    // resultTensor->GetSize() -- memory size  
    // resultTensor->GetBuffer() -- memory address  
}
```

In addition, the user needs to perform specific postprocessing on the data, for example, saving the model inference result in a file, or marking information such as the category and probability on an image. The following provides several common network parsing samples:

- Classification network result parsing

```
// Convert the output to AINeuralNetworkBuffer.  
shared_ptr<AINeuralNetworkBuffer> output_tensor =  
    static_pointer_cast<AINeuralNetworkBuffer>(output_data_vec[0]);  
// Convert the type of data in the buffer to float.  
float * result = (float *)output_tensor->GetBuffer();  
int label_index = 0;  
float max_value = 0.0;  
// Traverse to search for the largest category subscript and the corresponding confidence value.
```

```
for(int i=0; i< output_tensor->GetSize()/sizeof(float) ; i++)
{
    if(*(result + i) > max_value)
    {
        max_value = *(result + i);
        label_index = i;
    }
}
// Display the result.
printf("label index:%d, Confidence :%f\n", label_index, max_value);
```

- SSD network detection result parsing

```
// Generate data_num and data_bbox information.
IMAGE_HEIGHT = 300;
IMAGE_WIDTH = 300;

// The box_num result is a 4-byte float32 number, indicating that N boxes are detected on the
network.
std::shared_ptr<hiiai::AI NeuralNetworkBuffer> output_data_num =
std::static_pointer_cast<hiiai::AI NeuralNetworkBuffer>(output_data_vec[1]);

// The box_data result is shape (200, 7). The data type is float32.
std::shared_ptr<hiiai::AI NeuralNetworkBuffer> output_data_bbox =
std::static_pointer_cast<hiiai::AI NeuralNetworkBuffer>(output_data_vec[0]);
/*
|-----0-----1-----2-----3-----4-----5-----6---|
| image_id| Label | score | xmin | ymin | xmax | ymax |-----bbox1
| image_id| Label | score | xmin | ymin | xmax | ymax |-----bbox2
|-----|
*/
Use the first N boxes.
```

- Faster-RCNN detection network result parsing

```
// Generate data_num and data_bbox information, including 32 int32 numbers that indicate the
number of boxes for each detected target.
std::shared_ptr<hiiai::AI NeuralNetworkBuffer> output_data_num =
std::static_pointer_cast<hiiai::AI NeuralNetworkBuffer>(output_data_vec[0]);
/*
|--1---2---3---4---5-----32-----|
| 0 | 0 | 1 | 2 | 0 | ..... | 0 |
|-----|
*/
Indicates that label3 has one box, and label4 has two boxes. A label does not contain the background.

// Generate box_data. The dimensions are (32, 608, 8). (Note: The output dimensions are only an
example, which depends on the actual model.)
std::shared_ptr<hiiai::AI NeuralNetworkBuffer> output_data_bbox =
std::static_pointer_cast<hiiai::AI NeuralNetworkBuffer>(output_data_vec[1]);
/*
|-----|
| xmin | ymin | xmax | ymax | score | reserve | reserve | reserve |-----bbox1
| xmin | ymin | xmax | ymax | score | reserve | reserve | reserve |-----bbox2
label1 |
|
| xmin | ymin | xmax | ymax | score | reserve | reserve | reserve |-----bbox608
|-----|
.
.
.
|-----|
| xmin | ymin | xmax | ymax | score | reserve | reserve | reserve |-----bbox1
| xmin | ymin | xmax | ymax | score | reserve | reserve | reserve |-----bbox2
label32 |
|
| xmin | ymin | xmax | ymax | score | reserve | reserve | reserve |-----bbox608
|-----|

box[i, j, 0] indicates xmin of the jth box of the ith class.
box[i, j, 1] indicates ymin of the jth box of the ith class.
box[i, j, 2] indicates xmax of the jth box of the ith class.
```

```
box[i, j, 3] indicates ymax of the jth box of the kth class.
box[i, j, 4] indicates the score of the jth box of the kth class.
*/
```

3.3.7 Memory Management

3.3.7.1 Memory Management APIs Provided by the Native Language

The native language (C/C++) provides the **malloc**, **free**, **memcpy**, **memset**, **new**, and **delete** APIs for memory management. You can manage and control the lifecycle of memory allocated by using these APIs. If the memory to be allocated is less than 256 KB, memory management APIs provided by the native language and those provided by the Matrix module show similar performance. Therefore, you are advised to use a memory management API provided by the native language to simplify programming.

The following code shows how to use memory management APIs provided by the native language:

```
// Use malloc to alloc buffer
unsigned char* inbuf = (unsigned char*)malloc( fileLen );
// free buffer
free(inbuf);
inbuf = nullptr;
```

3.3.7.2 Memory Management APIs Provided by Matrix

Matrix provides a set of C/C++ APIs for allocating and freeing memory, including **HIAI_DMalloc/HIAI_DFree** and **HIAI_DVPP_DMalloc/HIAI_DVPP_DFree**. Among these APIs, **HIAI_DMalloc** and **HIAI_DFree** are used to allocate memory and transfer data from the host to the device side by working with **SendData**. While **HIAI_DVPP_DMalloc** and **HIAI_DVPP_DFree** are used to allocate memory for the DVPP on the device side. You can call the **HIAI_DMalloc/HIAI_DFree** and **HIAI_DVPP_DMalloc/HIAI_DVPP_DFree** APIs to allocate memory to reduce copy operations and save time.

API Description

[Table 3-2](#) describes the functions of the **HIAI_DMalloc/HIAI_DFree** and **HIAI_DVPP_DMalloc/HIAI_DVPP_DFree** APIs.

Table 3-2 API description

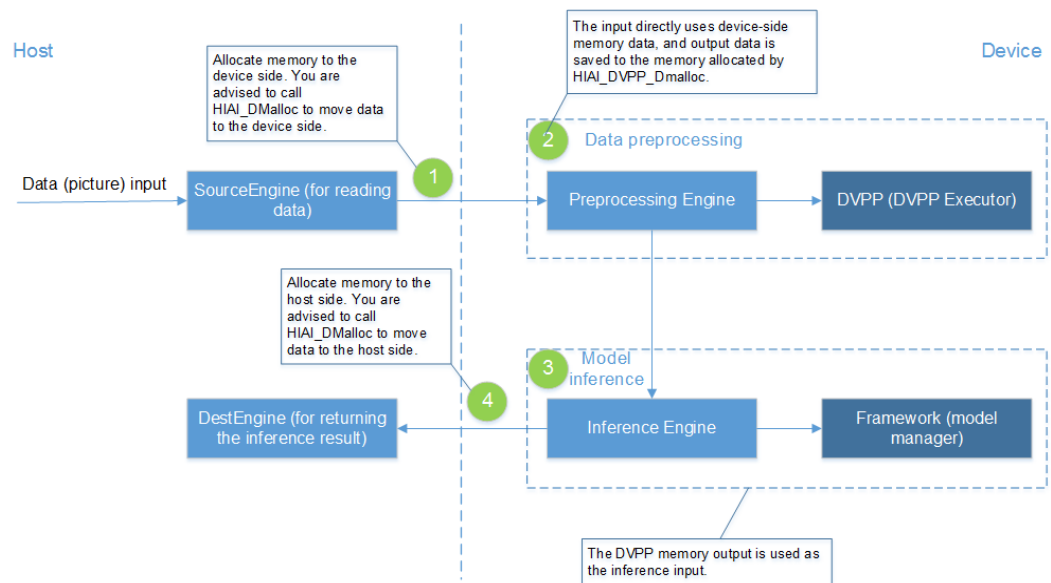
API Name	Function
HIAIMemory::HIAI_DMalloc (for C++ only)	Allocates memory. The memory to be allocated is similar to common memory but offers better performance in cross-side transmission (host-device/device-host) and model inference.

API Name	Function
HIAIMemory::HIAI_DFree (for C++ only)	Frees the memory allocated by HIAIMemory::HIAI_DMAlloc. This API is used together with HIAIMemory::HIAI_DMAlloc. When calling the HIAIMemory::HIAI_DMAlloc API, you can set flag to MEMORY_ATTR_AUTO_FREE. In this case, if data is sent to the peer end by calling the SendData API, you do not need to call the HIAIMemory::HIAI_DFree API and the allocated memory is automatically freed after the program is complete. However, if the SendData API is not called, or the SendData API is called but data fails to be sent to the peer end, you need to call HIAIMemory::HIAI_DFree to free the allocated memory.
HIAI_DMAlloc (for C/C++)	Allocates memory. The memory to be allocated is similar to common memory but offers better performance in cross-side transmission (host-device/device-host) and model inference.
HIAI_DFree (for C/C++)	Frees the memory allocated by HIAI_DMAlloc. This API is used together with HIAI_DMAlloc. When calling the HIAI_DMAlloc API, you can set flag to MEMORY_ATTR_AUTO_FREE. In this case, if data is sent to the peer end by calling the SendData API, you do not need to call the HIAI_DFree API and the allocated memory is automatically freed after the program is complete. However, if the SendData API is not called, or the SendData API is called but data fails to be sent to the peer end, you need to call HIAI_DFree to free the allocated memory.
HIAIMemory::HIAI_DVPP_DMAlloc (for C++ only)	Allocates memory for the DVPP on the device.
HIAIMemory::HIAI_DVPP_DFree (for C++ only)	Frees the memory allocated by the HIAIMemory::HIAI_DVPP_DMAlloc API.

API Name	Function
HIAI_DVPP_DMAlloc (for C/C++)	Allocates memory for the DVPP on the device.
HIAI_DVPP_DFree (for C/C++)	Frees the memory allocated by the HIAI_DVPP_DMAlloc API.

API Calling Process

Figure 3-7 API calling process



The usage of APIs in **Figure 3-7** is described as follows:

- The memory allocated by using the **HIAI_DMAlloc** or **HIAIMemory::HIAI_DMAlloc** API can be used in end-to-end data transmission and model inference, but cannot be used by the DVPP. The data transmission efficiency and performance can be improved by calling the **HIAI_DMAlloc** or **HIAIMemory::HIAI_DMAlloc** API and using the **HIAI_REGISTER_SERIALIZE_FUNC** macro that serializes or deserializes user-defined data types.

Allocating memory by using the **HIAI_DMAlloc** or **HIAIMemory::HIAI_DMAlloc** API has the following advantages:

- The allocated memory can be directly used by host-device communication (HDC) module for data transmission to avoid data copy between the Matrix module and HDC.
 - You can use the allocated memory for zero-copy inference to reduce data copy time.
- The memory allocated by using the **HIAI_DVPP_DMAlloc** or **HIAIMemory::HIAI_DVPP_DMAlloc** API can be used by the DVPP. After being used by the DVPP, data in the memory can be transparently transmitted to the inference model. If model inference is not required, data in the memory

allocated by using the **HIAI_DVPP_DMAlloc** API can be directly sent back to the host.

- The memory allocated by using the **HIAI_DMAlloc**, **HIAIMemory::HIAI_DMAlloc**, **HIAI_DVPP_DMAlloc**, and **HIAIMemory::HIAI_DVPP_DMAlloc** APIs is compatible with memory management APIs provided by the native language. It can be used as common memory, but cannot be freed by using APIs such as **free** and **delete**. Generally, memory allocated by using the **HIAI_DMAlloc**, **HIAIMemory::HIAI_DMAlloc**, **HIAI_DVPP_DMAlloc**, and **HIAIMemory::HIAI_DVPP_DMAlloc** APIs needs to be freed by calling **HIAI_DFree**, **HIAIMemory::HIAI_DFree**, **HIAI_DVPP_DFree**, and **HIAIMemory::HIAI_DVPP_DFree**, respectively.

When calling the **HIAI_DMAlloc** or **HIAIMemory::HIAI_DMAlloc** API, you can set **flag** to **MEMORY_ATTR_AUTO_FREE**. In this case, if data is sent to the peer end by calling the **SendData** API, you do not need to call the **HIAIMemory::HIAI_DFree** API and the allocated memory is automatically freed after the program is complete. However, if the **SendData** API is not called, or the **SendData** API is called but data fails to be sent to the peer end, you need to call **HIAIMemory::HIAI_DFree** to free the memory.

- The memory allocated by using the **HIAI_DVPP_DMAlloc** or **HIAIMemory::HIAI_DVPP_DMAlloc** API meets the requirements of the DVPP. Therefore, when the resources are limited, you are advised to use these APIs only for the DVPP.

Precautions for API Usage

When allocating memory by using **HIAI_DMAlloc** or **HIAIMemory::HIAI_DMAlloc**, pay attention to the following issues about memory management:

- When memory to be automatically freed is allocated for host-device or device-host data sending, if a smart pointer is used, Matrix automatically frees the memory, so the destructor specified by the smart pointer must be null. If the pointer is not a smart pointer, Matrix automatically frees the memory.
- When allocating memory to be manually freed for host-device or device-host data transmission, if a smart pointer is used, you need to set the destructor to **HIAI_DFree** or **HIAIMemory::HIAI_DFree**. If the pointer is not a smart pointer, you need to call **HIAI_DFree** or **HIAIMemory::HIAI_DFree** to free the memory after data transmission is complete.
- When allocating memory to be automatically freed, do not call the **SendData** API to send data for multiple times.
- When memory to be manually freed is allocated for host-device or device-host data sending, do not reuse the data in the memory before the memory is freed. If the memory is used for host-host or device-device data sending, the data in the memory can be reused before the memory is freed.
- When memory to be manually freed is allocated, if the **SendData** API is called to asynchronously send data, data in the memory cannot be modified after being sent.

If the **HIAI_DVPP_MAlloc** or **HIAIMemory::HIAI_DVPP_DMAlloc** API is called to allocate memory for device-host data transmission, you need to call the

HIAI_DVPP_DFree or **HIAIMemory::HIAI_DVPP_DFree** API to manually free the memory, because the **HIAI_DVPP_MAlloc** or **HIAIMemory::HIAI_DVPP_DMAlloc** API does not automatically free the memory. If a smart pointer is used to store the allocated memory address, the destructor must be set to **HIAI_DVPP_DFree** or **HIAIMemory::HIAI_DVPP_DFree**.

API Calling Example

(1) When the performance optimization solution is used to transmit data, the data transmit API must be manually serialized and deserialized.

// Note: The serialization function is used at the transmit end and the deserialization function is used at the receive end. Therefore, you are advised to register this function with both transmit and receive ends.

// Data structure

```
typedef struct
{
    uint32_t left_offset = 0;
    uint32_t right_offset = 0;
    uint32_t top_offset = 0;
    uint32_t bottom_offset = 0;
    // The serialize function is used to serialize a struct.
    template <class Archive>
    void serialize(Archive & ar)
    {
        ar(left_offset, right_offset, top_offset, bottom_offset);
    }
} crop_rect;
// Registers the type of data sent by the engine.
typedef struct EngineTransNew
{
    std::shared_ptr<uint8_t> trans_buff = nullptr; // Transfers buffer
    uint32_t buffer_size = 0; // Transfers buffer size
    std::shared_ptr<uint8_t> trans_buff_extend = nullptr;
    uint32_t buffer_size_extend = 0;
    std::vector<crop_rect> crop_list;
    // The serialize function is used to serialize a struct.
    template <class Archive>
    void serialize(Archive & ar)
    {
        ar(buffer_size, buffer_size_extend, crop_list);
    }
} EngineTransNewT;

// Serialization function
/**
 * @ingroup hiaiengine
 * @brief GetTransSearPtr, // Serializes the trans data.
 * @param [in]: data_ptr // Struct pointer
 * @param [out]: struct_str // Struct buffer
 * @param [out]: data_ptr // Struct data pointer buffer
 * @param [out]: struct_size // Struct size
 * @param [out]: data_size // Struct data size
 */
void GetTransSearPtr(void* data_ptr, std::string& struct_str,
    uint8_t*& buffer, uint32_t& buffer_size)
{
    EngineTransNewT* engine_trans = (EngineTransNewT*)data_ptr;
    uint8_t* dataPtr = (uint8_t*)engine_trans->trans_buff.get();
    uint32_t dataLen = engine_trans->buffer_size;
    uint8_t* dataPtr_extend = (uint8_t*)engine_trans->trans_buff_extend.get();
    uint32_t dataLen_extend = engine_trans->buffer_size_extend;
    std::shared_ptr<uint8_t> data = ((EngineTransNewT *)data_ptr)->trans_buff;
    std::shared_ptr<uint8_t> data_extend = ((EngineTransNewT *)data_ptr)->trans_buff_extend;
    // Obtains the struct buffer and size.
    buffer_size = dataLen + dataLen_extend;
    buffer = (uint8_t*)engine_trans->trans_buff.get();
    engine_trans->trans_buff = nullptr;
    engine_trans->trans_buff_extend = nullptr;
```

```

// Serialization
std::ostream outputStr;
cereal::PortableBinaryOutputArchive archive(outputStr);
archive((*engine_trans));
struct_str = outputStr.str();

((EngineTransNewT*)data_ptr)->trans_buff = data;
((EngineTransNewT*)data_ptr)->trans_buff_extend = data_extend;
}
void deleteNothing(void* ptr)
{
    // do nothing
}

// Deserialization function
/**
 * @ingroup hiaiengine
 * @brief GetTransSearPtr, // Deserializes the trans data.
 * @param [in]: ctrl_ptr // Struct pointer
 * @param [in]: data_ptr // Struct data pointer
 * @param [out]: std::shared_ptr<void> // Struct pointer assigned to the engine
 */
std::shared_ptr<void> GetTransDearPtr(
    const char* ctrlPtr, const uint32_t& ctrlLen,
    const uint8_t* dataPtr, const uint32_t& dataLen)
{
    if(ctrlPtr == nullptr) {
        return nullptr;
    }
    std::shared_ptr<EngineTransNewT> engine_trans_ptr = std::make_shared<EngineTransNewT>();
    // Assigns a value to engine_trans_ptr.
    std::istringstream inputStream(std::string(ctrlPtr, ctrlLen));
    cereal::PortableBinaryInputArchive archive(inputStream);
    archive((*engine_trans_ptr));
    uint32_t DataLen = engine_trans_ptr->buffer_size;
    uint32_t DataLen_extend = engine_trans_ptr->buffer_size_extend;
    if(dataPtr != nullptr) {
        (engine_trans_ptr->trans_buff).reset((const_cast<uint8_t*>(dataPtr)), hiai::HIAIMemory::HIAI_DFree);
        (engine_trans_ptr->trans_buff_extend).reset((const_cast<uint8_t*>(dataPtr + DataLen)),
deleteNothing);
    }
    return std::static_pointer_cast<void>(engine_trans_ptr);
}

// Registers EngineTransNewT.
HIAI_REGISTER_SERIALIZE_FUNC("EngineTransNewT", EngineTransNewT, GetTransSearPtr, GetTransDearPtr);

(2) When sending data, you can use only the registered data types. Use HIAI_DMalloc to allocate memory
to optimize performance.
Note: When transferring data from the host to the device side, you are advised to use HIAI_DMalloc to
optimize transmission efficiency. The data size supported by the HIAI_DMalloc API ranges from 0 bytes to
(256 MB – 96 bytes). If the data size exceeds this range, use the malloc API to allocate memory.
// Allocates the data memory by calling the HIAI_DMalloc API. The value 10000 indicates the delay in
microseconds, that is, if the memory space is insufficient, the program waits 10000 ms.
HIAI_StatusT get_ret = HIAIMemory::HIAI_DMalloc(width*align_height*3/2,(void*)&align_buffer, 10000);

// Sends data. After the SendData API is called, the HIAI_DFree API does not need to be called. The
value 10000 indicates the delay.
graph->SendData(engine_id_0, "TEST_STR", std::static_pointer_cast<void>(align_buffer), 10000);

```

3.4 Building a Project

Build a project to generate executable files of the application project and the engine library files on which project running depends. For details, see Tutorial > Application Development in the *Mind Studio User Manual*.

3.5 Running a Project

After the project is built, you can run it on the Ascend AI processor to obtain the computation result. For details, see Tutorial > Application Development in the *Mind Studio User Manual*.

4 FAQs

- 4.1 What Do I Do If a Core Dump Occurs in the Multi-Thread Environment When the `std::cout` and `printf` Are Used Together?
- 4.2 What Do I Do If Memory Is Exhausted Due to an Excessive Engine Buffer?
- 4.3 How Do I Configure `ai_config` During Engine::Init Overloading?
- 4.4 What Do I Do If Data Errors Occur When the Receive Memory of Multiple Engines Correspond to the Same Memory Buffer?
- 4.5 How Do I View the Requirements of Offline Models on the Arrangement of Input Image Data?
- 4.6 How Do I Configure `thread_num` to Meet Multi-Channel Video Decoding Requirements?
- 4.7 What Do I Do If the `thread_num` Configuration Is Incorrect During Multi-Channel Video Decoding?
- 4.8 What Do I Do If the Engine Thread Fails to Exit Due to an Engine Implementation Code Error on the Host Side?
- 4.9 What Do I Do If the Engine Thread Fails to Exit Due to an Engine Implementation Code Error on the Device Side?
- 4.10 Adding a Third-Party Library
- 4.11 What Do I Do If Memory Allocation For Other Graphs Fails After the First Graph Is Destroyed in the Single-Process, Multi-Graph Scenario?

4.1 What Do I Do If a Core Dump Occurs in the Multi-Thread Environment When the `std::cout` and `printf` Are Used Together?

Rule

Do not use **`std::cout`** and **`printf`** together. Otherwise, a core dump may occur in multi-thread environments.

printf and **std::cout** are the functions in the standard C and C++ respectively. **printf** does not have buffers while **std::cout** has. They also differ in the time when the standard output should be locked.

- **printf**: The standard output is locked before it is processed.
- **std::cout**: The standard output is locked only when it is printed.

The two functions differ slightly in timing. However, in a multi-thread environment, even a minor timing difference may cause many problems. Therefore, the mixed use of the two functions may cause unpredictable errors. For example, the printed output is not as expected, or even the internal buffer may overflow, leading to a core dump.

Counter Example

```
#include<stdio.h>
#include<iostream.h>
int main(int argc, char* argv[])
{
    int j=0;
    for(j=0;j<5;j++)
    {
        cout<<"j=";
        printf("%d\n",j);
    }
    return 0;
}
```

The output of the preceding code may be as follows:

```
1
2
3
4
j=j=j=j=j=
```

This is clearly against the expected result. The reason is that the standard stream output of the **std::cout** function has a buffer. If the buffer is not cleared in time and the output function of another system is used, the two output functions may be incompatible, leading to unexpected errors. Therefore, you are advised to check the compatibility with the standard print output in the code and use unified print output.

Correct Example

```
#include<stdio.h>
int main(int argc, char* argv[])
{
    int j=0;
    for(j=0;j<5;j++)
    {
        printf("j=");
        printf("%d\n",j);
    }
    return 0;
}
```

or

```
#include<stdio.h>
#include<iostream.h>
int main(int argc, char* argv[])
{
    int j=0;
```

```
for(j=0;j<5;j++)
{
    cout<<"j=";
    cout<<j<<"\n";
}
return 0;
}
```

4.2 What Do I Do If Memory Is Exhausted Due to an Excessive Engine Buffer?

Symptom

On a developer board, 16 inference processes are used to process 1080p images concurrently. As a result, the memory is used up, and the processes exit after the memory allocation fails.

Cause Analysis

To prevent jitter, an engine queue size is 200 by default. In the preceding symptom, if the queue is full, 600 MB (3 MB x 200) memory is used. If the queues of 16 inference processes (assuming that each process has three engines) become full at the same time, 29 GB memory is required, far exceeding the upper limit (8 GB). As a result, the memory is used up, and the processes exit after the memory allocation fails.

Solution

Set the engine queue size based on the service jitter, size of data received by the engine, and system memory to ensure that services are not affected.

A too small value will cause a timeout when the **SendData** interface of the engine is called to send data. A too large value will cause memory exhaustion.

Step 1 Log in to the DDK server as the DDK installation user.

Step 2 Modify the graph configuration file and adjust the engine queue size **queue_size**.

Assume that the project path is **\$HOME/tools/projects/Custom_Engine**.

vi \$HOME/tools/projects/Custom_Engine/test_data/config/sample.prototxt

```
engines {
  id: 1001
  engine_name: "DvppEngine"
  side: DEVICE
  thread_num: 1
  queue_size: 40 // Queue size. The default value is 200.
}
```

Step 3 Save the settings and exit.

----End

4.3 How Do I Configure ai_config During Engine::Init Overloading?

During the implementation of **Engine::Init** overloading, you need to assign a value to the parameter of the **AIConfig** type in the graph configuration file. The following are several typical scenarios:

- Scenario 1: **Engine::Init** overloading during data engine development

```
HIAI_StatusT DataInput::Init(const hiai::AIConfig& config, const
std::vector<hiai::AIModelDescription>& model_desc) {
    HIAI_ENGINE_LOG(HIAI_IDE_INFO, "[DataInput] Start init!");

    //read the config of dataset
    for (int index = 0; index < config.items_size(); ++index) {
        const ::hiai::AIConfigItem& item = config.items(index);
        std::string name = item.name();
        if (name == "target") {
            target_ = item.value();
        } else if (name == "path") {
            path_ = item.value();
        }
    }
    //get the dataset image info
    MakeDatasetInfo();

    HIAI_ENGINE_LOG(HIAI_IDE_INFO, "[DataInput] End init!");
    return HIAI_OK;
}
```

In this case, **ai_config** of the data engine in the graph configuration file must be set to the path of the dataset file.

```
engines {
  id: 611
  engine_name: "DataInput"
  side: HOST
  thread_num: 1
  so_name: "./libHost.so"
  ai_config {

    items {
      name: "path"
      value: "/home/lyz1/AscendProjects/myApp2/resource/data/"
    }

    items {
      name: "target"
      value: "RC"
    }
  }
}
```

- Scenario 2: **Engine::Init** reloading during the preprocessing engine development

```
HIAI_StatusT ImagePreProcess::Init(const hiai::AIConfig& config, const
std::vector<hiai::AIModelDescription>& modelDesc) {
    HIAI_ENGINE_LOG(HIAI_IDE_INFO, "[ImagePreProcess] Start init!");
    dvppConfig_ = std::make_shared<DvppConfig>();
    if (dvppConfig_ == nullptr || dvppConfig_.get() == nullptr) {
        HIAI_ENGINE_LOG(HIAI_IDE_ERROR, "[ImagePreProcess] Failed to call make_shared for
DvppConfig.");
        return HIAI_ERROR;
    }
    //get config from ImagePreProcess Property setting of user.
    std::stringstream ss;
```

```
for (int index = 0; index < config.items_size(); ++index) {
    const ::hiai::AIConfigItem& item = config.items(index);
    std::string name = item.name();
    ss << item.value();
    if ("resize_height" == name) {
        ss >> dvppConfig->resize_height;
    } else if ("resize_width" == name) {
        ss >> dvppConfig->resize_width;
    }
    ss.clear();
}

if (DVPP_SUCCESS != CreateDvppApi(pidvppapi_)) {
    HIAI_ENGINE_LOG(HIAI_IDE_ERROR, "[ImagePreProcess] Failed to call CreateDvppApi.");
    return HIAI_ERROR;
}
HIAI_ENGINE_LOG(HIAI_IDE_INFO, "[ImagePreProcess] End init!");
return HIAI_OK;
}
```

In this case, **ai_config** of the preprocessing engine in the graph configuration file must be set to the target length and width after data preprocessing.

```
engines {
    id: 814
    engine_name: "ImagePreProcess"
    side: DEVICE
    thread_num: 1
    so_name: "./libDevice.so"
    ai_config {

        items {
            name: "resize_width"
            value: "224"
        }

        items {
            name: "resize_height"
            value: "224"
        }
    }
}
```

- Scenario 3: **Engine::Init** reloading during the model inference engine development

```
HIAI_StatusT FrameworkerEngine::Init(const hiainf::AIConfig& config,
    const std::vector<hiainf::AIModelDescription>& model_desc)
{
    hiainf::AIStatus ret = hiainf::SUCCESS;
    // init ai_model_manager_
    if (nullptr == ai_model_manager_)
    {
        ai_model_manager_ = std::make_shared<hiainf::AIModelManager>();
    }
    std::cout<<"FrameworkerEngine Init"<<std::endl;
    HIAI_ENGINE_LOG("FrameworkerEngine Init");

    for (int index = 0; index < config.items_size(); ++index)
    {

        const ::hiai::AIConfigItem& item = config.items(index);
        // loading model
        if(item.name() == "model_path")
        {
            const char* model_path = item.value().data();
            std::vector<hiainf::AIModelDescription> model_desc_vec;
            hiainf::AIModelDescription model_desc;
            model_desc.set_path(model_path);
            model_desc.set_key("");
            model_desc_vec.push_back(model_desc);
```

```
ret = ai_model_manager->Init(config, model_desc_vec);

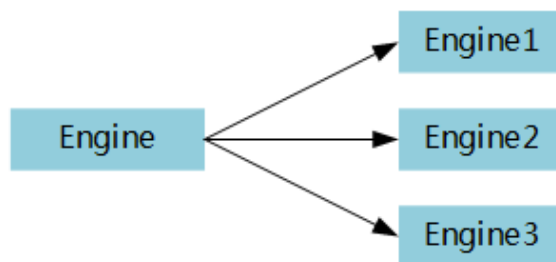
if (hiai::SUCCESS != ret)
{
    HIAI_ENGINE_LOG(this, HIAI_AI_MODEL_MANAGER_INIT_FAIL, "[DEBUG] fail to init
ai_model");
    return HIAI_AI_MODEL_MANAGER_INIT_FAIL;
}
}
}
HIAI_ENGINE_LOG("FrameworkerEngine Init success");
return HIAI_OK;
}
```

In this case, **ai_config** of the model inference engine in the graph configuration file must be set to the model file path.

```
engines {
  id: 1003
  engine_name: "FrameworkerEngine"
  so_name: "./libFrameworkerEngine.so"
  side: DEVICE
  thread_num: 1
  ai_config{
    items{
      name: "model_path"
      value: "./test_data/model/resnet18.om"
    }
  }
}
```

4.4 What Do I Do If Data Errors Occur When the Receive Memory of Multiple Engines Correspond to the Same Memory Buffer?

When the receive memory of multiple engines correspond to the same memory buffer, if the buffer of an engine is modified, data errors occur on other engines.



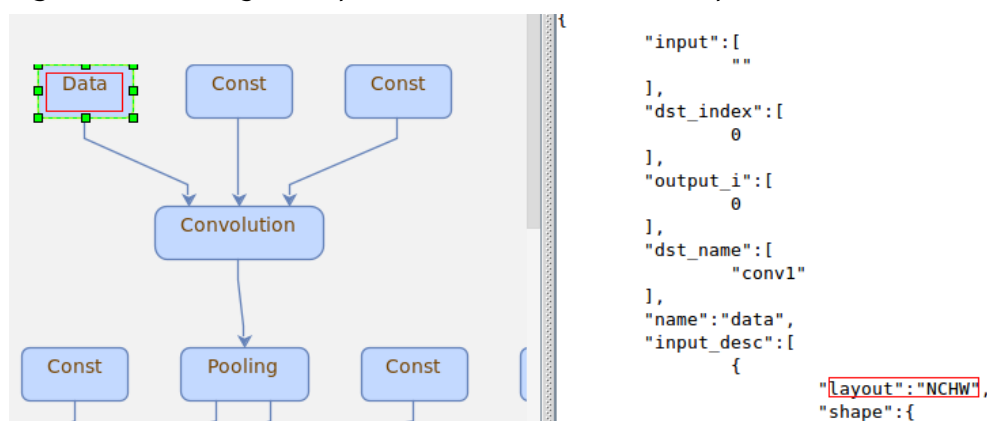
You are advised to perform a deep copy before modifying the buffer content.

```
std::shared_ptr<MyType> tmp_arg = std::static_pointer_cast<MyType>(arg0);
// Perform the deep copy.
std::shared_ptr<MyType> input_arg = std::make_shared<MyType>();
memcpy(input_arg.get(), tmp_arg.get(), sizeof(MyType));
// Modify data.
input_arg->data = 1;
```

4.5 How Do I View the Requirements of Offline Models on the Arrangement of Input Image Data?

- Step 1** Log in to **Mind Studio**.
- Step 2** Choose **Tools > View Model**.
- Step 3** Select a converted model file (for example, **resnet18.om**), and click **Open**.
- Step 4** In the displayed window, view the input information of the data operator, as shown in **Figure 4-1**.

Figure 4-1 Viewing the input information of the data operator



----End

4.6 How Do I Configure thread_num to Meet Multi-Channel Video Decoding Requirements?

When Matrix is used to orchestrate the application process and DVPP is used to decode multiple channels of input video streams, each VDEC engine must fixedly correspond to one video stream to ensure the data sequence. Otherwise, the VDEC engines of DVPP cannot decode video streams.

When multiple video streams are decoded, multiple implementation modes may be available to ensure the data sequence of each frame in the video streams. You are advised to use the following configurations:

- In the graph configuration file, configure multiple VDEC engines in the graph segment. One video stream corresponds to one engine.
- In the graph configuration file, set **thread_num** to **1** in the VDEC engine segment. One engine corresponds to one thread.

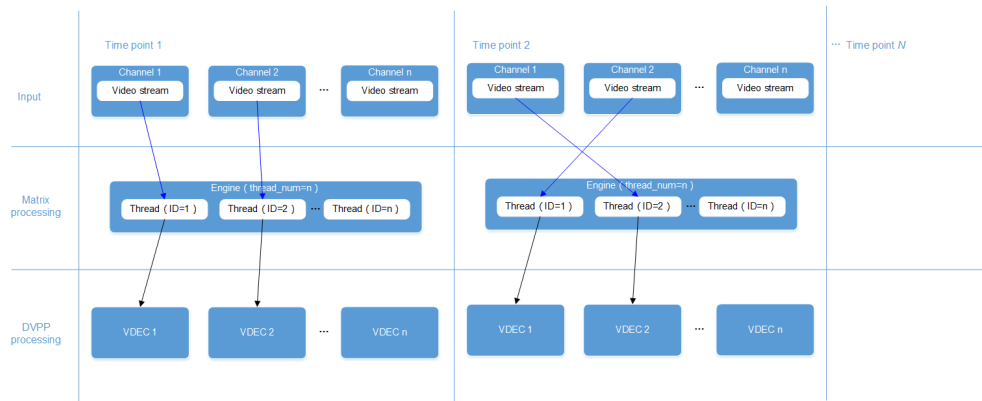
NOTE

One graph (one thread) can correspond to a maximum of 16 video streams.

If only one VDEC engine is configured in the graph configuration file and **thread_num** is set to **n** (the value of **n** is the number of video stream channels)

during multi-channel video stream decoding, video streams in Matrix may be processed in different threads at different time points. However, in DVPP, one VDEC engine requires one fixed channel of video stream data. One channel of video stream data corresponds to one thread. In this case, the data sequence of each frame in the video streams may not be ensured during video decoding, as shown in [Figure 4-2](#).

Figure 4-2 VDEC process



4.7 What Do I Do If the thread_num Configuration Is Incorrect During Multi-Channel Video Decoding?

Symptom

When the multi-channel video decoding application is running and the graph is destroyed, the graph running process on the host side exits abnormally.

- Log in to the host server as the **HwHiAiUser** user and view the **/var/dlog/device-*/device-*.logid** file. The log records that the VDEC decoding task fails.
[ERROR] DVPP(24531,graph_1161):2019-10-11-08:04:22.978.699 [VDEC] [EventHandler:336] [T34] cannot find video in global video queue! [ERROR] DVPP(24531,graph_1161):2019-10-11-08:04:22.978.826 [VDEC] [event_process:2042] [T34] generate_command_done failed
- Log in to the host server as the **HwHiAiUser** user and view the **/var/dlog/host-0/host-0_*.log** file. The log records that the operation times out.
[ERROR] HIAIENGINE(15959,hiai_dvpp_test):2019-10-11-08:05:13.533.616 destroy_timeout_ms = 30000,[...DestroyGraph], Msg: destroy timeout

When the application is started again, log in to the host server as the **HwHiAiUser** user and view the **/var/dlog/device-*/device-*.logid** file. The log records that the graph process has been occupied:

In the preceding command, **1161** is the graph ID, and **1783** is the ID of the process used to run the graph, which are subject to the actual requirements.

[ERROR] HIAIENGINE(2685,matrixdaemon):2019-10-28-23:56:33.545.880 /home/HwHiAiUser/matrix/1161 has already existed, and pid:1783 is alive and using it. Fail, [CreateDeviceDirAndPidFile], Msg: directory already used in another file failed

Cause Analysis

The VDEC module on the device side cannot exit properly and is suspended. As a result, the graph with the same ID fails to be created.

Check the graph configuration file. In the **engine** configuration segment of the VDEC module, the value of **thread_num** is greater than **1**. When checking the application code logic, **CreateVdecApi** is called only once for multi-channel video decoding to obtain a VDEC handle. As a result, the same VDEC handle is used for multi-channel decoding, and the process is suspended.

Solution

Step 1 Stop the graph process based on the process ID in the **device-id_*.log** file.

The following is a command example. Replace **1783** with the actual process ID.
`kill -9 1783`

Step 2 Modify the graph file by referring to the recommended configuration in [4.6 How Do I Configure thread_num to Meet Multi-Channel Video Decoding Requirements?](#)

Step 3 Recompile and run the application.

----End

4.8 What Do I Do If the Engine Thread Fails to Exit Due to an Engine Implementation Code Error on the Host Side?

Symptom

A running application fails to exit.

Cause Analysis

- Log in to the host side as the **HwHiAiUser** user and run the **ps -elf | grep application name** command to obtain the process ID (PID) of the application.
For example, in the following figure, **main** indicates the application name, and **4817** indicates the PID of the application.

Figure 4-3 Command example

```
$ ps -elf | grep main
0 S HwHiAiU+ 4817 4703 0 80 0 - 364797 futex_ 09:34 pts/0 00:00:00 ./main 0 0
0 S HwHiAiU+ 5155 5088 0 80 0 - 5027 pipe_w 09:34 pts/1 00:00:00 grep main
```

- Run the following command based on the PID (for example, **4817**) in **1** to check the list of engine threads that have not exited:
`for i in $(ls /proc/4817/task); do grep Name /proc/$i/status | awk '{print $2}'; done | grep -E "e[0-9]+_w[0-9]+"`

An engine thread is named after **e + EngineId + _w + Thread ID**. In the engine section of the graph configuration file, if **thread_num** is set to **1**, the

thread ID is **0**. If **thread_num** is set to a value greater than **1**, the thread ID starts from 0.

Figure 4-4 Command output example

```
$ for i in $(ls /proc/4817/task); do grep Name /proc/$i/status | awk '{print $2}' ; done | grep -E "e[0-9]_w[0-9]+"
e1100_w0
e1101_w0
e1102_w0
e1103_w0
e1104_w0
e1105_w0
e1106_w0
e1107_w0
e1108_w0
e1109_w0
e2100_w0
```

3. Check whether the following incorrect code logic (included but not limited to) exists in the implementation code (implementation code of the **HIAI_IMPL_ENGINE_PROCESS** macro) of the engines that do not exit.

- Infinite loop

Sample code:

```
HIAI_IMPL_ENGINE_PROCESS("face_register", FaceRegister, INPUT_SIZE)
{
    //HIAI_ENGINE_LOG(HIAI_ENGINE_RUN_ARGS_NOT_RIGHT, "xxx PROCESS ...");
    while (1) {
        if (!DoRegisterProcess()) {
            HIAI_ENGINE_LOG(HIAI_ENGINE_RUN_ARGS_NOT_RIGHT,
                "DoRegisterProcess failed.");
        }
    }

    //HIAI_ENGINE_LOG(HIAI_ENGINE_RUN_ARGS_NOT_RIGHT, "xxx PROCESS 1...");

    return HIAI_OK;
}
```

- Blocking

On the socket that is not configured with the **O_NONBLOCK** mode, the **recv** interface is called to receive data, resulting in the blocking.

- Time-consuming operation, for example, long sleep

In the code logic, the code such as **sleep(500)** exists, whose execution takes a long time.

- Deadlock, including repeated locking of non-recursive locks, unlocking of some branches, and inconsistent locking sequence
- Process suspension. For details, see [4.7 What Do I Do If the thread_num Configuration Is Incorrect During Multi-Channel Video Decoding?](#).

Solution

- Step 1** Log in to the host side as the **HwHiAiUser** user and stop the process based on the PID (for example, **4817**) in **1**.

```
kill -9 4817
```

- Step 2** Modify the implementation code of the **HIAI_IMPL_ENGINE_PROCESS** macro of the corresponding engine.

If code of the infinite loop cannot be deleted due to service requirements, you are advised to add code for creating a thread to the **Engine::Init** interface and define

the loop condition as a variable (for example, **flag**). Whether to exit the loop depends on the variable value. Change the **flag** value in the destructor function of the engine to ensure that the thread normally exits. For example, when the **flag** value is **0**, the thread exits the loop.

Step 3 Recompile and run the application.

----End

4.9 What Do I Do If the Engine Thread Fails to Exit Due to an Engine Implementation Code Error on the Device Side?

Symptom

When an application runs for the first time, no error is reported throughout the running.

When the application is started again, log in to the host server as the **HwHiAiUser** user and view the **/var/dlog/device-*/device-id_*.log** file. The log records that the graph process has been occupied:

In the preceding command, **1160** is the graph ID, and **2823** is the ID of the process used to run the graph. Their values are subject to the actual requirements.

```
[ERROR] HIAIENGINE(2685,matrixdaemon):2019-10-28-23:56:33.545.880 /home/HwHiAiUser/matrix/1160
has already existed, and pid:2823 is alive and using it. Fail,
[CreateDeviceDirAndPidFile], Msg: directory already used in another file failed
```

Cause Analysis

1. If this process still exists after the application running on the host side has been ended for 3s or more, log in to the device side as the **HwHiAiUser** user and run the following command based on the process ID (for example, **2823**) of the graph in the **device-id_*.log** file to check the list of engine threads that have not exited:

```
for i in $(ls /proc/2823/task); do grep Name /proc/$i/status | awk '{print $2}'; done | grep -E
"e[0-9]+_w[0-9]+"
```

An engine thread is named after **e + EngineId + _w + Thread ID**. In the engine section of the graph configuration file, if **thread_num** is set to **1**, the thread ID is **0**. If **thread_num** is set to a value greater than **1**, the thread ID starts from **0**.

Figure 4-5 Command output example

```
[HwHiAiUser@none] ~]$ for i in $(ls /proc/2823/task); do grep Name /proc/$i/status | awk '{print $2}'; done | grep -E "e[0-9]+_w[0-9]+"
```

```
e1200_w0
e1201_w0
e1202_w0
e1203_w0
e1204_w0
e1205_w0
e1206_w0
e1207_w0
e1208_w0
e1209_w0
e1300_w0
e1301_w0
e1302_w0
e1303_w0
e1304_w0
e1305_w0
e1306_w0
```

NOTE

The SSH service on the device side is disabled by default, which has to be enabled by calling **dsmi_set_user_config**. For details about the API usage, see the *DSMI API Reference*.

2. Check whether the following incorrect code logic (included but not limited to) exists in the implementation code (implementation code of the **HIAI_IMPL_ENGINE_PROCESS** macro) of the engines that do not exit.

- Infinite loop

Sample code:

```
HIAI_IMPL_ENGINE_PROCESS("face_register", FaceRegister, INPUT_SIZE)
{
    //HIAI ENGINE LOG(HIAI ENGINE RUN ARGS NOT RIGHT, "xxx PROCESS ...");
    while (1) {
        if (!DoRegisterProcess()) {
            HIAI_ENGINE_LOG(HIAI_ENGINE_RUN_ARGS_NOT_RIGHT,
                "DoRegisterProcess failed.");
        }
    }

    //HIAI_ENGINE_LOG(HIAI_ENGINE_RUN_ARGS_NOT_RIGHT, "xxx PROCESS 1...");
    return HIAI_OK;
}
```

- Blocking

On the socket that is not configured with the **O_NONBLOCK** mode, the **recv** interface is called to receive data, resulting in the blocking.

- Time-consuming operation, for example, long sleep

In the code logic, the code such as **sleep(500)** exists, whose execution takes a long time.

- Deadlock, including repeated locking of non-recursive locks, unlocking of some branches, and inconsistent locking sequence
- Process suspension. For details, see [4.7 What Do I Do If the thread_num Configuration Is Incorrect During Multi-Channel Video Decoding?](#).

Solution

- Step 1** Stop the graph process based on the process ID in the **device-id_*.log** file. Log in to the device side as the **HwHiAiUser** user to stop the process.

The following is a command example. Replace **2823** with the actual process ID.

```
kill -9 2823
```

- Step 2** Modify the implementation code of the **HIAI_IMPL_ENGINE_PROCESS** macro of the corresponding engine.

If code of the infinite loop cannot be deleted due to service requirements, you are advised to add code for creating a thread to the **Engine::Init** interface and define the loop condition as a variable (for example, **flag**). Whether to exit the loop depends on the variable value. Change the **flag** value in the destructor function of the engine to ensure that the thread normally exits. For example, when the **flag** value is **0**, the thread exits the loop.

Step 3 Recompile and run the application.

----End

4.10 Adding a Third-Party Library

The following describes how to add a third-party library during application development by assuming that the device-side engine depends on the third-party library **libjsoncpp.so**.

Step 1 Save the third-party library to **lib/device** in the project directory.

The third-party library must be stored in the preceding path so that it can be copied to the running environment during project running.

Step 2 Edit the **src/graph.config** file in the project directory.

```
engines {
  id: 1001
  engine_name: "HelloWorldEngine"
  so_name: "./lib/device/libjsoncpp.so"
  so_name: "./libDevice.so"
  side: DEVICE
  thread_num: 1
}
```

Step 3 Edit the **src/CMakeLists.txt** file in the project directory.

Table 4-1 Description of parameters in the CMakeLists file

Parameter	Description
include_directories	Path of the header file of the third-party library, such as the following information in bold: # Header path include_directories(. \$ENV{DDK_PATH}/include/inc/ \$ENV{DDK_PATH}/include/third_party/protobuf/include \$ENV{DDK_PATH}/include/third_party/jsoncpp \$ENV{DDK_PATH}/include/libc_sec/include Common DataInput ImagePreProcess SaveFilePostProcess MindInferenceEngine)
link_directories	Directory to which the third-party library file is to be added, such as the following information in bold: # add device lib path link_directories(\$ENV{NPU_DEV_LIB} ../lib/device)
target_link_libraries	Name of the library to be linked to the target file. The first argument in this command indicates that created target files are generated by running the add_executable() and add_library() commands. The following arguments indicate the names of the .lib files without suffixes. For details, see the following information in bold: target_link_libraries(main matrixdaemon mmpa pthread dl rt) target_link_libraries(Host matrixdaemon hiai_common) target_link_libraries(Device Dvpp_api Dvpp_jpeg_decoder Dvpp_jpeg_encoder Dvpp_vpc idedaemon hiai_common jsoncpp)

----End

4.11 What Do I Do If Memory Allocation For Other Graphs Fails After the First Graph Is Destroyed in the Single-Process, Multi-Graph Scenario?

Symptom

In the scenario where multiple graphs are created in a single process, if the first graph is destroyed, memory allocation using **DMalloc** for other graphs might temporarily fail on the host.

The error code returned by the **DMalloc** API is **16847020** (**HIAI_GRAPH_MEMORY_POOL_STOPPED**), indicating that the memory pool is stopped.

```
HIAI_DMalloc failed graphId:10306 time:1581666533376.000000 ret:16847020
HIAI_DMalloc failed graphId:10305 time:1581666533376.000000 ret:16847020
HIAI_DMalloc failed graphId:10312 time:1581666533376.000000 ret:16847020
HIAI_DMalloc failed graphId:10307 time:1581666533376.000000 ret:16847020
HIAI_DMalloc failed graphId:10304 time:1581666533376.000000 ret:16847020
HIAI_DMalloc failed graphId:10315 time:1581666533376.000000 ret:16847020
HIAI_DMalloc failed graphId:10315 time:1581666533376.000000 ret:16847020
```

Cause Analysis

Graphs of the same process on the host share one memory pool. The status of the memory pool maintained by the first graph applies.

When the first graph is destroyed, the memory pool is stopped, leading to the **DMalloc** fault. After the first graph is completely destroyed, the second graph becomes the first graph, and the memory pool is restored.

Solution

You are advised not to destroy the first graph when an exception occurs. If necessary, a retry mechanism can be introduced for the error code **HIAI_GRAPH_MEMORY_POOL_STOPPED** returned by calling to **HIAI_DMalloc** or **HIAIMemory::HIAI_DMalloc**. The sample code is as follows.

```
int cnt = 0;
while (cnt < 5) {
    if (HIAI_GRAPH_MEMORY_POOL_STOPPED == HIAI_DMalloc(dataSize, timeOut, flag)) {
        usleep(10000);
        cnt++;
    } else {
        break;
    }
}
```

5 Appendix

- [5.1 Description of the Multi-Card Multi-Chip Scenario for Atlas 300](#)
- [5.2 Description of the Multi-Card Multi-Chip Scenario for Atlas 200 DK](#)
- [5.3 Change History](#)

5.1 Description of the Multi-Card Multi-Chip Scenario for Atlas 300

Currently, the supported development scenarios include single-chip, multi-chip, and multi-chip task splitting scenarios. You can select an application scenario based on actual requirements.

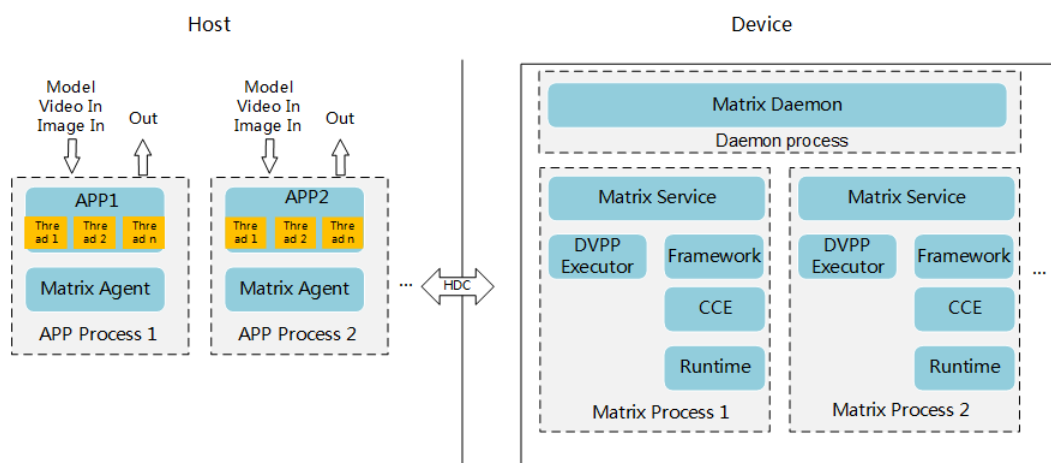
NOTE

For the user application processes on the host side, you can create either one graph or multiple graphs in serial mode for one thread as required. It is recommended that one graph be created for each thread.

One graph corresponds to one Matrix process on the device side.

Single-Chip Scenario

Figure 5-1 Single-chip Scenario

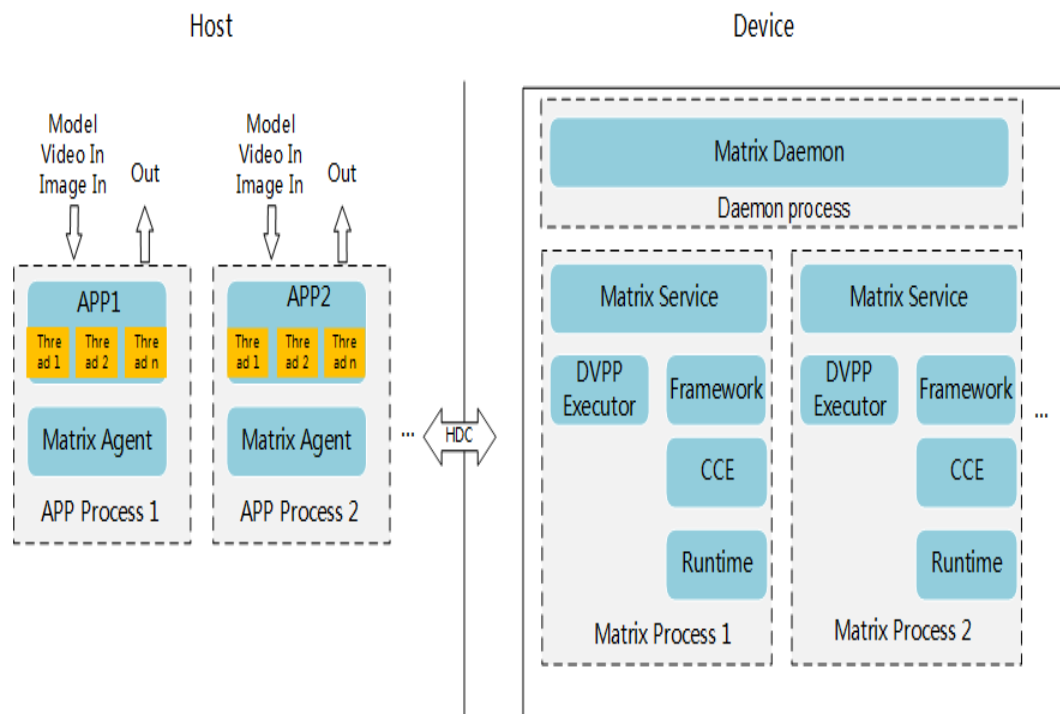


In the single-chip scenario, one Ascend 310 processor is configured on the host side. This scenario includes the following sub-scenarios:

- Single application, single thread, single chip: A single thread of an application initiates an inference task, which is pushed to the device for execution. The Matrix server creates a process for each inference task. For example, the task on the device corresponding to thread 1 on application 1 is Matrix process 1.
- Single application, multiple threads, single chip: Multiple threads of an application initiate inference tasks respectively, which are pushed to the device for execution. The Matrix server creates a process for each inference task. For example, the task on the device corresponding to thread 1 on application 1 is Matrix process 1.
- Multiple applications, single thread for each application, single chip: The inference task of application 1 corresponds to the task of Matrix process 1 on the device, and the inference task of application 2 corresponds to the task of Matrix process 2 on the device.
- Multiple applications, multiple threads for each application, single chip: The inference task of thread 1 on application 1 corresponds to the task of Matrix process 1 on the device, and the inference task of thread 2 on application 2 corresponds to the task of Matrix process 2 on the device.

Multi-Chip Scenario

Figure 5-2 Multi-chip Scenario



In the multi-chip scenario, multiple PCIe cards are configured on the host, with each equipped with multiple Ascend 310 processors. This scenario includes the following sub-scenarios:

- Single application, multiple threads, multiple chips: Multiple threads on an application initiate a complete inference task respectively. To fully utilize the concurrent processing performance of multiple devices, the inference task of thread 1 on application 1 runs on Ascend 310 of device 1, and the corresponding task is Matrix process 1. The inference task of thread 2 on application 1 runs on Ascend 310 of device 2. The rule applies. Threads and devices are independent from each other. The application is responsible for processing the handling result of multiple threads.
- Multiple applications, single thread for each application, multiple chips: The inference task of application 1 runs on device 1, and the corresponding inference task is Matrix process 1. The inference task of application 2 runs on device 2.
- Multiple applications, multiple threads for each application, and multiple chips: The inference task of application 1 thread 1 runs on device 1, the corresponding inference task is Matrix process 1, and the inference task of application 2 thread 2 runs on device 2.

Multi-Chip Task Splitting

In the task splitting scenario, each thread independently initiates a complete inference task, and an inference flow contains multiple network algorithm models. Users can set different models to run on various chips. For example, algorithm 1 is deployed and runs on PCIe1-Device1, algorithm 2 is deployed and runs on PCIe1-Device2, and algorithm 3 is deployed and runs on PCIe2-Device n . A task is completed by the cooperation between multiple chips. If only one chip is used, the task does not need to be split.

The multi-chip task splitting scenario includes the following sub-scenarios. For details, see [Figure 5-2](#).

- Single application, single thread, multiple chips
- Single application, multiple threads, multiple chips
- Multiple applications, single thread for each application, multiple chips
- Multiple applications, multiple threads for each application, multiple chips

5.2 Description of the Multi-Card Multi-Chip Scenario for Atlas 200 DK

The Atlas 200 DK scenario includes the following sub-scenarios. You can develop applications for a specific scenario based on actual requirements.

- Single application, single thread: An application starts a thread, which initiates an inference task.
- Single application, multiple threads: An application starts multiple threads, and each thread initiates an inference task separately.
- Multiple applications, single thread for each application: Each application starts a thread, and each thread initiates an inference task separately.
- Multiple applications, multiple threads for each application: Multiple applications start multiple threads, and each thread initiates an inference task separately.

5.3 Change History

Date	Description
2020-05-30	This issue is the first official release.